

RĪGAS VALSTS TEHNIKUMS

DATORIKAS NODAĻA

Izglītības programma: Programmēšana

KVALIFIKĀCIJAS DARBS

**“Tīmekļa lietojumprogramma pārtikas
pārvaldībai un ēdienu recepšu ieteikšanai”**

Paskaidrojošais raksts 64.lpp.

Audzēknis:

Alise Evelīna Toča, DP4-1

Prakses vadītāji:

Ilona Demčenko, Monta Montvida

Nodaļas vadītājs:

Normunds Barbāns

Rīga, 2026

ANOTĀCIJA

Tīmekļa lietojumprogramma pārtikas krājumu pārvaldībai un ēdiena recepšu ieteikšanai ir izstrādāta, lai nodrošinātu ērtu mājsaimniecības pārtikas produktu uzskaiti, derīguma termiņu kontroli un recepšu atlasī, balstoties uz pieejamajām sastāvdaļām. Sistēma apvieno vairākas funkcionalitātes vienotā risinājumā un ir pielāgota vairāku lietotāju sadarbībai. Izstrādes laikā tika izmantotas tādas tehnoloģijas kā HTML5, CSS3 (SCSS), JavaScript (ES6) ar Vue.js 3.5, Inertia.js, Ziggy.js, PHP 8.3 ar Laravel 12, MySQL 8.3, kā arī Laravel Filament administrācijas paneļa izveidei. Papildus tika izmantoti Docker un Laravel Sail izstrādes videi un Mailtrap e-pastu testēšanai..

Kvalifikācijas darbs ietver ievadu, uzdevumu nostādni, prasību specifikāciju, uzdevuma risināšanas līdzekļu izvēles pamatojumu, programmatūras modelēšanas un projektēšanas aprakstu, datu struktūru aprakstu, lietotāja ceļvedi un secinājumus. Ievadā tiek pamatota tēmas aktualitāte un problēmas būtība. Uzdevumu nostādnē definēti sistēmas mērķi un uzdevumi. Prasību specifikācijā aprakstītas funkcionālās un nefunkcionālās prasības. Risinājuma līdzekļu izvēles pamatojumā izskaidrota izmantoto tehnoloģiju izvēle. Modelēšanas un projektēšanas daļā attēlota sistēmas arhitektūra un darbības principi. Datu struktūru aprakstā sniegta datubāzes struktūra. Lietotāja ceļvedī aprakstīta sistēmas lietošana.

Kvalifikācijas darbs sastāv no 94. lappusēm, kurās ietilpst 59 attēli, 27 tabulas un 8 pielikumi.

ANNOTATION

The web application for food inventory management and recipe recommendation is developed to provide convenient tracking of household food products, expiration date control, and recipe suggestions based on available ingredients. The system integrates multiple functionalities into a single solution and is designed for collaboration between multiple users. During development, technologies such as HTML5, CSS3 (SCSS), JavaScript (ES6) with Vue.js 3.5, Inertia.js, Ziggy.js, PHP 8.3 with Laravel 12, MySQL 8.3, and Laravel Filament for the administration panel were used. Additionally, Docker and Laravel Sail were used for the development environment, and Mailtrap for email testing.

The qualification paper includes an introduction, task definition, requirements specification, justification of the chosen development tools, software modeling and design description, data structure description, user guide, and conclusions. The introduction explains the relevance of the topic and the problem addressed. The task definition outlines the system's objectives. The requirements specification describes functional and non-functional requirements. The justification section explains the choice of technologies. The modeling and design section presents the system architecture and operation principles. The data structure section describes the database structure. The user guide provides instructions for using the system.

The qualification paper consists of 94 pages, including 59 images, 27 tables, and 8 appendices.

SATURS

IEVADS.....	5
1. UZDEVUMA NOSTĀDNE.....	7
2. PRASĪBU SPECIFIKĀCIJA.....	9
2.1. Ieejas un izejas informācijas apraksts.....	9
2.1.1. Ieejas informācijas apraksts.....	9
2.1.2. Izejas informācijas apraksts.....	11
2.2. Funkcionālās prasības.....	11
2.3. Nefunkcionālās prasības.....	20
3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS.....	23
4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA.....	25
4.1. Sistēmas struktūras modelis.....	25
4.1.1. Sistēmas arhitektūra.....	25
4.1.2. Sistēmas ER modelis.....	26
4.2. Funkcionālais sistēmas modelis.....	28
4.2.1. Datu plūsmu modelis.....	28
5. DATU STRUKTŪRAS APRAKSTS.....	33
6. LIETOTĀJA CEĻVEDIS.....	41
6.1. Sistēmas prasības aparatūrai un programmatūrai.....	41
6.2. Sistēmas instalācija un palaišana.....	42
6.2.1. Sistēmas piekļūšana lokālā vidē.....	42
6.2.2. Sistēmas piekļūšana hostingā.....	46
6.3. Programmas apraksts.....	47
7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA.....	62
NOBEIGUMS.....	65
INFORMĀCIJAS AVOTI.....	66
PIELIKUMI.....	67
Receptes PDF izdrukas paraugs.....	67
Pievienošanās mājsaimniecībai uzaicinājuma e-pasta paziņojums.....	68
Paroles maiņas e-pasta paziņojums.....	69
Filtrēšanas piemērs recepu skatā.....	70
Kārtošanas piemērs pēc atsauksmju skaita administrācijas paneļa skatā.....	71
Meklēšanas piemērs recepšu skatā.....	72
Datubāzes tabulu saišu shēma.....	73
Sistēmas pirmkods.....	74

IEVADS

Mūsdienās mājsaimniecību steigpilnā un arvien aizņemtāka dzīves ritma dēļ kļūst aizvien svarīgāk izmantot vienotu digitālu risinājumu, kas ļauj ērti pārvaldīt ikdienas pārtikas krājumus un to pielietojumu dažādās ēdiena receptēs. Gan ģimenes, gan studenti, kā arī personas, kas dzīvo patstāvīgi, saskaras ar problēmām izsekot pārtikas produktu derīguma termiņiem, izvērtēt, kādi pārtikas produkti jau atrodas virtuvē, un saprast, kā tos lietderīgi izmantot ēdienreižu plānošanā. Aktualitāti pastiprina tas, ka pārtikas izšķērdēšana joprojām ir būtiska problēma, un daļa mājsaimniecību to piedzīvo tieši informācijas trūkuma vai neefektīvas uzskaites dēļ. Pašreiz tirgū pieejamās lietotnes koncentrējas galvenokārt uz atsevišķām funkcijām, piemēram, ēdiena receptu pārvaldību vai tikai sastāvdaļu pārvaldību, taču trūkst risinājumu, kas apvienotu abus, nodrošinot pārtikas produktu uzskaiti, derīgumu termiņu kontroli un ēdiena receptu ieteikumus atkarībā no esošajām sastāvdaļām. Lai atrisinātu šo problēmu, tiks izstrādāta tīmekļa lietojumprogramma pārtikas pārvaldībai un ēdiena receptu ieteikšanai. Šāds risinājums palīdzētu samazināt nevajadzīgus tērīšus, veicināt pārdomātu pārtikas izmantošanu un atvieglotu ikdienas ēdienreižu organizēšanu.

Tirgus izpētes laikā tika secināts, ka pašreizējā tirgū nav pilnvērtīgas sistēmas, kas vienlaikus nodrošinātu pārtikas produktu uzskaiti un iespēju vairākiem lietotājiem to pārvaldīt, derīgumu termiņu kontroli un ēdiena receptu ieteikšanu atkarībā no esošām sastāvdaļām. Liela daļa cilvēku joprojām pārtikas produktus pārvalda ar pierakstiem mobilajos bloknos, ledusskapja sarakstiem vai pat papīra lapiņām, kas var viegli pazust.

Viens no risinājumiem ir tīmekļa vietne un lietotne SuperCook, kas ļauj lietotājiem ievadīt sastāvdaļas un saņemt ēdiena receptes, kuras ir iespējams pagatavot no pieejamajiem pārtikas produktiem. Risinājums nodrošina plašu pārtikas produktu klāstu, ļauj pievienot virtuves iekārtas un uztura ierobežojumus, kā arī nodrošina bagātīgu ēdiena receptu klāstu no dažādu valstu virtuvēm. SuperCook galvenā priekšrocība ir tās intuitīvā un automātiskā ēdiena receptu meklēšana pēc lietotāja atzīmētajām sastāvdaļām. Tomēr būtisks trūkums ir tas, ka SuperCook koncentrējas tikai uz ēdiena receptu meklēšanu un nenodrošina lietotāja pārtikas produktu uzskaiti vai to derīguma termiņu kontroli. Lietojumprogramma neuzglabā informāciju par to, cik pārtikas produktu ir atlicis, kā arī nepiedāvā iespēju pārvaldīt virtuves krājumus vairākām personām vienlaikus. Šī ierobežotā funkcionalitāte padara lietotni mazāk piemērotu mājsaimniecībām, kas vēlas centralizētu un precīzu pārtikas krājumu pārvaldību.

Savukārt PantryCheck ir lietotne, kas vērsta uz pārtikas produktu uzskaiti un derīgumu termiņu kontroli. Tā piedāvā vairākas funkcijas, kas ļauj pārtikas krājumus strukturēt, atjaunināt un sekot to patēriņam. Viena no būtiskākajām priekšrocībām ir iespēja pievienot pārtikas produktus, izmantojot svītrkoda skenēšanu, kas ievērojami paātrina jaunu pārtikas produktu pievienošanu pārvaldības sistēmai. Turklāt lietotne automātiski apkopo un vizuāli attēlo informāciju par pārtikas produkta derīguma termiņu, sniedzot brīdinājumus, kad kāda no vienībām tuvojas izbeigšanās datumam, kā arī pārtikas produkti tiek sakārtoti pēc tā kategorijas un atrašanās vietas virtuvē. Bet, neskatoties uz priekšrocībām, risinājumam ir vairāki trūkumi - netiek piedāvātas ēdiena receptes atkarībā no pārtikas krājumiem un svītrkodu skenēšanas funkcionalitāte Latvijā nav pilnvērtīgi izmantojama, jo ievērojama daļa vietējo pārtikas produktu nav atrodamā lietotnes datubāzē. Tas nozīmē, ka pārtikas produkti bieži vien jāpievieno manuāli, kas palēnina procesu un samazina uzskaites precizitāti.

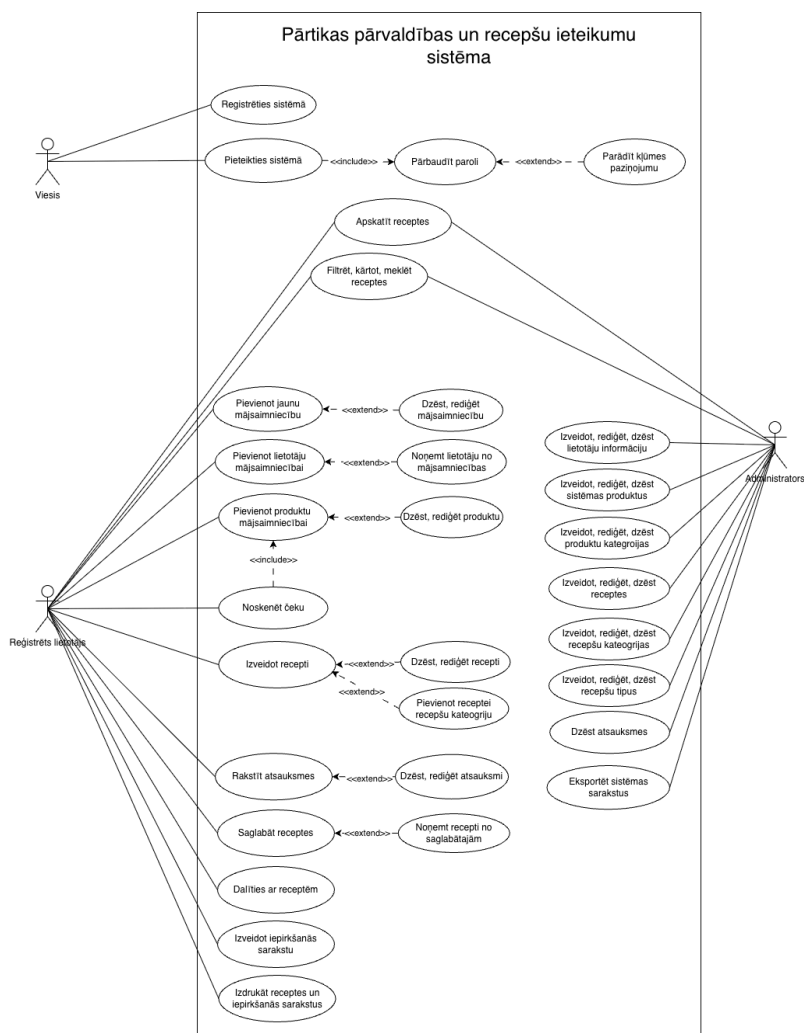
Pēc konkurentu piedāvājumu analīzes veikšanas var secināt, ka neviena no aplūkotajām sistēmām nav pilnībā piemērota mājtsaimniecību pārtikas krājumu pārvaldībai un ēdiena recepšu ieteikšanai. Papildu ierobežojums ir tas, ka svītrkodu atpazīšana vietējiem Latvijas pārtikas produktiem bieži vien nav pilnvērtīga, kas liek lietotājiem ievadīt datus manuāli. Abi risinājumi piedāvā tikai daļu no nepieciešamā funkcionalitātes kopuma, tādēļ tie nespēj pilnībā nodrošināt integrētu ikdienas pārtikas pārvaldības procesu.

Risinājums, kas tiks izstrādāts šī darba ietvaros, atšķirsies no konkurentu piedāvātajām sistēmām ar to, ka tas apvienos pārtikas krājumu uzskaiti, derīguma termiņu uzraudzību un ēdiena recepšu ieteikumus vienotā tīmekļa lietojumprogrammā. Piedāvātā sistēma būs optimizēta ērtai sadarbībai starp vairākiem mājtsaimniecības dalībniekiem. Šāda pieeja atvieglos ikdienas pārtikas organizēšanu, samazinās nevajadzīgu pārtikas izšķērdēšanu un nodrošinās tās funkcionalitātes, kas šobrīd tirgū pieejamajiem risinājumiem trūkst.

1. UZDEVUMA NOSTĀDNE

Kvalifikācijas darba uzdevums ir izveidot mājsaimniecības pārvaldības un ēdiena receptu meklēšanas sistēmu. Sistēmā nepieciešams realizēt iespēju glabāt un pārvaldīt datus par lietotāju mājsaimniecībām, tajās esošajiem pārtikas produktiem un to derīguma termiņiem, kā arī lietotāju izveidotām ēdiena receptēm un atsauksmēm. Reģistrētiem lietotājiem ir jābūt iespējai ērti piekļūt informācijai par savām mājsaimniecībām, pārtikas produktu krājumiem, ēdiena receptēm, iepirkumu sarakstiem un atsauksmēm, kā arī izmantot filtrēšanas, kārtošanas un meklēšanas iespējas, lai atrastu ēdiena receptes atbilstoši pieejamajiem mājsaimniecības pārtikas produktiem. Administratoram ir jānodrošina iespēja pārvaldīt un veikt informācijas izdrukas par sistēmas lietotājiem, ēdiena receptēm ar tās kategorijām un tiem, pārtikas produktiem un tā kategorijām, atsauksmēm. Sistēmā jāparedz arī iespēja viesiem reģistrēties un pieslēgties sistēmai.

Ir plānotas vairākas zemāk aprakstītas funkcijas (skat. 1.1.att.).



1.1.att. Lietojumgadījuma diagramma

- Administratoram būtu iespēja:
 - apskatīt, pievienot, dzēst un rediģēt informāciju par sistēmas lietotājiem;
 - apskatīt, pievienot, dzēst un rediģēt receptes;
 - apskatīt, izveidot, rediģēt, dzēst sistēmas pārtikas produktus;
 - apskatīt, pievienot, dzēst un rediģēt kategorijas, kas tiek izmantotas pārtikas produktu klasificēšanai;
 - apskatīt, pievienot, dzēst un rediģēt recepšu kategorijas, kas tiek izmantotas recepšu klasificēšanai;
 - apskatīt, pievienot, dzēst un rediģēt recepšu tipus, kas tiek izmantotas recepšu klasificēšanai;
 - apskatīt, dzēst atsauksmes;
- Reģistrētiem lietotājiem būtu iespēja:
 - pievienot, dzēst un rediģēt savas receptes;
 - pievienot, rediģēt, dzēst savām receptēm kategorijas un receptes veidus;
 - pievienot, dzēst un rediģēt savas mājsaimniecības;
 - pievienot, dzēst un rediģēt pārtikas produktus savās mājsaimniecībās (kopā ar to derīguma termiņiem un daudzumiem);
 - pievienot un dzēst citus lietotājus no savas mājsaimniecības
 - pievienot, rediģēt un dzēst savas atsauksmes;
 - izveidot un dzēst iepirkumu sarakstus un tā pārtikas produktus;
 - izveidot, dzēst receptes sarakstus un pievienot/ņņonemt receptes tajos.
 - meklēt, filtrēt un kārtot receptes pēc dažādiem kritērijiem, tai skaitā pēc sastāvdaļu atbilstības;
 - izdrukāt receptes un iepirkumu sarakstus.
- Viesiem būtu iespēja:
 - reģistrēties sistēmā;
 - pieslēgties sistēmā.

2. PRASĪBU SPECIFIKĀCIJA

2.1. Ieejas un izejas informācijas apraksts

2.1.1. Ieejas informācijas apraksts

Sistēmā ieejas informācija tiks iegūta no lietotāja ievades no tastatūras. Lietotājam no tastatūras būs iespējams ievadīt visu sistēmas informāciju.

1. Informācija par **lietotājiem** sastāvēs no sekojošajiem datiem.
 - Vārds - lietotāja pilnais vārds - rakstzīmju virkne (piem. Evelīna).
 - Lietotājvārds - lietotāja segvārds - rakstzīmju virkne, kura garums ir līdz 20 rakstzīmēm (piem. evelina_berz1).
 - E-pasts - lietotāja elektroniskā pasta adrese - rakstzīmju virkne (evelinaberzina@gmail.com).
 - Parole - rakstzīmju virkne ar kuru lietotājs autorizējas sistēmā - rakstzīmju virkne (piem. BerzEv!67).
2. Informācija par **mājsaimniecību** sastāvēs no sekojošajiem datiem.
 - Nosaukums - mājsaimniecības nosaukums - rakstzīmju virkne, kura garums ir līdz 30 rakstzīmēm (piem. Ģimenes ledusskapis).
3. Informācija par **pārtikas produktiem** sastāvēs no sekojošajiem datiem.
 - Nosaukums - pārtikas produkta nosaukums - rakstzīmju virkne, kuras garums ir līdz 50 rakstzīmēm (piem. Tomāts).
4. Informācija par **mājsaimniecības pārtikas produktiem** sastāves no sekojošajiem datiem.
 - Daudzums - pārtikas produkta daudzums - decimālskaitlis, kurš nav negatīvs (piem. 20,5).
 - Mērvienība - pārtikas produkta mērvienība - rakstzīmju virkne, kuras garums ir līdz 5 rakstzīmēm (piem. g, kg, ml, gab).
 - Derīguma termiņš - datums līdz kuram pārtikas produkts ir derīgs - datums (piem. 04.09.2027.)
5. Informācija par **pārtikas produktu kategorijām** sastāvēs no sekojošajiem datiem.
 - Nosaukums - kategorijas nosaukums - rakstzīmju virkne, kuras garums ir līdz 50 rakstzīmēm (piem. Dārzeni, piena produkti).
6. Informācija par **recepšu veidiem** sastāvēs no sekojošajiem datiem.

- Nosaukums - ēdienreizes nosaukums - rakstzīmju virkne, kuras garums ir līdz 50 rakstzīmēm (piem. Brokastis).
7. Informācija par **recepšu kategorijām** sastāvēs no sekojošajiem datiem.
- Nosaukums - veida nosaukums - rakstzīmju virkne, kuras garums ir līdz 50 rakstzīmēm (piem. veģetārie ēdieni, zupas, gaļas ēdieni).
8. Informācija par **receptēm** sastāvēs no sekojošajiem datiem.
- Nosaukums - receptes nosaukums - rakstzīmju virkne, kura garums ir līdz 40 rakstzīmēm (piem. Karbonādes ar siera cepurīti).
 - Attēls - receptes fotoattēls - grafiskais fails.
 - Saturs - receptes apraksts - garš receptes aprakstošs teksts.
 - Redzamība - receptes redzamība - rakstzīmju virkne (privāta, mājsaimniecības ietvaros, publiska).
 - Sagatavošanās laiks - receptes sagatavošanas ilgums – laiks, kas jāpavada pirms receptes gatavošanas (piem. 20min).
 - Gatavošanas laiks - receptes gatavošanas ilgums - laiks, kas jāpavada gatavojot recepti (piem 1h 5min).
 - Porcijas - ēdiena daudzums uz cilvēkiem - skaitlis cik daudz porcijas ietver recepte (piem. 4).
 - Pagatavošanas soļi - receptes pagatavošanas soļi - teksts ar pagatavošanas soļiem.
9. Informācija par **recepšu pārtikas produktiem** sastāvēs no sekojošajiem datiem.
- Daudzums - pārtikas produkta daudzums - decimālskaitlis, kurš nav negatīvs (piem. 20,5).
 - Mērvienība - pārtikas produkta mērvienība - rakstzīmju virkne, kuras garums ir līdz 5 rakstzīmēm (piem. g, kg, ml, gab).
10. Informācija par **mapēm** sastāvēs no sekojošajiem datiem.
- Nosaukums - receptes saraksta nosaukums - rakstzīmju virkne, kuras garums ir līdz 30 rakstzīmēm (piem. “Svētkiem”).
11. Informācija par **atsauksmēm** sastāvēs no sekojošajiem datiem.
- Saturs - lietotāja komentārs - īss teksts, kura garums ir līdz 500 rakstzīmēm.
 - Attēls - lietotāja izveidotās receptes ēdiens - grafiskais fails.
 - Novērtējums - lietotāja receptes novērtējums - skaitlis no 0 līdz 5 (piem. 4).
12. Informācija par **iepirkumu saraksta vienībām** sastāvēs no sekojošajiem datiem.

- Produkts - lietotāja pievienotais pārtikas produkts - teksts ar pārtikas produktu (piem. 400g vistas filejas).

2.1.2. Izejas informācijas apraksts

1. **PDF izdruka** ar recepti (sk. 1.pielikumu), kur dokumenta kreisajā pusē ir virsraksts ar receptes nosaukumu, īsi zem tā seko receptes autors, aprēķinātais vidējais receptes lietotāju novērtējums. Dokumenta pamatdaļā jāatrodas receptes attēlam, saturam, porciju skaitam, pagatavošanas laikam, nepieciešamie pārtikas produkti receptes izveidei un pagatavošanas soļi.
2. **E-pasta paziņojums** (sk. 2.pielikumu) ar uzaicinājumi saiti, lai pievienotos mājsaimniecībā. Tēmā ir paziņojums, ka esat uzaicināts uz attiecīgās mājsaimniecības nosaukumu un sistēmas nosaukums. Saturā ir apraksts un saite uz sistēmu.
3. **E-pasta paziņojums** (sk. 3. pielikumu) par paroles maiņu. Tēmā ir paziņojums par attiecīgā lietotāja konta paroles maiņu. Saturā ir apraksts un saite uz sistēmas tīmekļa vietni, kur lietotājam ir iespēja iestatīt jauno paroli.
4. **Filtrēšana** (sk. 4.pielikumu) visām administratora paneļa tabulām, kā arī recepšu filtrēšanai pēc to tipiem, kategorijām un pieejamības.
5. **Kārtošana** (sk. 6.pielikumu) administratora paneļa datu kārtošanai augošā, dilstošā secībā.
6. **Meklēšana** (sk. 6.pielikumu) sistēmā ir recepšu skatā, kur ir iespējams meklēt pieejamās receptes atkarībā no to nosaukuma.

2.2. Funkcionālās prasības

1. Lietotāju pieteikšanās sistēmā (sk. 2.1.tabulu).

2.1.tabula

Lietotāja pieteikšanās funkcijas detalizēts apraksts

Identifiaktors	PS 2.2.1
Ievads	
Funkcija paredzēta, lai lietotājs varētu autentificēties sistēmā un piekļūt sistēmas funkcionalitātei atbilstoši savām tiesībām.	
Ievaddati	
1. Lietotājvārds vai e-pasta adrese - obligāts lauks. 2. Parole - obligāts lauks.	

Apstrāde
1. Sistēma pārbauda, vai ievadītais lietotājvārds vai e-pasts eksistē datu bāzē. 2. Sistēma pārbauda ievadītās paroles atbilstību lietotājam. 3. Ja ievaddati ir pareizi, lietotājs tiek autorizēts.
Izvaddati
1. Lietotājs tiek novirzīts uz sava konta galveno sadaļu. 2. Kļūdas gadījumā tiek izvadīts paziņojums.
Kļūdas paziņojumi
1. Ievadīti nepareizi autorizācijas dati.
Izsaucamās prasības
1. Var veikt jebkurš lietotājs ar piekļuvi internetam.

2. Lietotāju piesaiste mājsaimniecībai (sk. 2.2.tabulu).

2.2.tabula

Lietotāja mājsaimniecības piesaistes funkcijas detalizēts apraksts

Identifikators	PS 2.2.2
Ievads	
Funkcija nodrošina citu lietotāju pievienošanu mājsaimniecībai, izmantojot e-pasta uzaicinājumu.	
Ievaddati	
1. Mājsaimniecības identifikators - obligāts lauks. 2. Vēlamā lietotāja e-pasts - obligāts lauks.	
Apstrāde	
1. Sistēma pārbauda vai lietotājam, kurš ievadīja datus, ir mājsaimniecības saimnieka tiesības. 2. Sistēma pārbauda ievaddatu atbilstību prasībām. 3. Sistēma ģenerē atpazīšanas zīmi un to pievieno uzaicinājuma saitei. 4. Saite tiek nosūtīta uz uzaicinātā lietotāja e-pastu.	
Izvaddati	
1. E-pasts ar uzaicinājuma saiti. 2. Lietotājs pievienots mājsaimniecībai.	
Kļūdas paziņojumi	
1. Lietotājam nav mājsaimniecības saimnieka tiesības. 2. Obligātie lauki nav aizpildīti. 3. Lauka formāts neatbilst prasībām. 4. Nevar pievienot lietotāju - lietotājs jau ir pievienots mājsaimniecībai. 5. Saite nav derīga - atpazīšanas zīmei ir iestājusies noildze.	

<i>Izsaucamās prasības</i>
1. Pieejama tikai reģistrētiem lietotājiem ar saimnieka tiesībām mājsaimniecībā.

3. Mājsaimniecības pārtikas produktu uzskaitē (sk. 2.3.tabulu).

2.3.tabula

Mājsaimniecības produkta uzskaites funkcijas detalizēts apraksts

<i>Identifikators</i>	PS 2.2.3
<i>Ievads</i>	
Funkcija paredzēta pārtikas produktu uzskaitē mājsaimniecības dalībniekiem.	
<i>Ievaddati</i>	
1. Mājsaimniecības identifikators - obligāts lauks. 2. Pārtikas produkta nosaukums - obligāts lauks. 3. Daudzums - obligāts lauks. 4. Mērvienība - obligāts lauks. 5. Derīguma termiņš - neobligāts lauks.	
<i>Apstrāde</i>	
1. Sistēma pārbauda ievaddatu formātu. 2. Sistēma ielasa norādītās mājsaimniecības esošo pārtikas produktu sarakstu. 3. Ja pārtikas produkts ar tādu pašu nosaukumu un derīguma termiņu jau ir mājsaimniecībā, jauns ieraksts netiek izveidots un eksistējošā ieraksta daudzums tiek saskaitīts kopā ar ievadīto daudzumu. 4. Citādāk tiek pievienots jauns pārtikas produkts mājsaimniecībai ar lietotāja ievaddatiem. 5. Mājsaimniecības pārtikas produktu saraksts tiek atjaunināts.	
<i>Izvaddati</i>	
1. Atjaunināts mājsaimniecības pārtikas produktu saraksts. 2. Jauns pārtikas produkta ieraksts mājsaimniecībā.	
<i>Kļūdas paziņojumi</i>	
1. Obligātie lauki nav aizpildīti. 2. Lauka formatējums neatbilst prasībām.	
<i>Izsaucamās prasības</i>	
1. Pieejama mājsaimniecības dalībniekiem.	

4. Receptu meklēšana pēc mājsaimniecībā pieejamajiem pārtikas produktiem (sk. 2.4.tabulu).

Mājsaimniecības produkta uzskaites funkcijas detalizēts apraksts.

Identifiaktors	PS 2.2.4
Ievads	
Funkcija ļauj lietotājam piedāvāt receptes, balstoties uz mājsaimniecībā esošajiem pārtikas produktiem.	
Ievaddati	
1. Mājsaimniecības pārtikas produktu saraksts - obligāts lauks. 2. Receptes saraksti - obligāts lauks.	
Apstrāde	
1. Sistēma salīdzina mājsaimniecības pārtikas produktu sarakstu ar receptes pārtikas produktu sarakstiem. 2. Tiek aprēķināta receptes atbilstība daļot pārtikas produktus, kas atbilst receptei, ar kopējo pārtikas produktu skaitu receptē. 3. Tiek atjaunots receptes atbilstības stabiņš mājsaimniecības produktiem. 4. Sistēma atlasa visatbilstošākās receptes un kārtot tās pēc mājsaimniecību produktu atbilstības.	
Izvaddati	
1. Recepte atbilstības stabiņa vērtība. 2. Atrasto recepšu saraksts.	
Kļūdas paziņojumi	
1. Nav atrastas receptes, kas atbilst pieejamajiem produktiem.	
Izsaucamās prasības	
1. Pieejama autentificētiem lietotājiem ar piesaistītu mājsaimniecību.	

5. Receptes pieejamība lietotāju grupām (sk. 2.5.tabulu.).

Receptes pieejamības funkcijas detalizēts apraksts.

Identifiaktors	PS 2.2.5
Ievads	
Funkcija nodrošina, ka pieeja receptei ir tikai atbilstošo lietotāju grupām.	
Ievaddati	
1. Receptes redzamība (privāts, mājsaimniecības ietvaros, publisks) - obligāts lauks. 2. Receptes autora identifikators - obligāts lauks. 3. Lietotāja identifikators - obligāts lauks.	
Apstrāde	
1. Sistēma pārbauda ievaddatu formātu.	

Apstrāde
2. Sistēma pārbauda ievaddatu formātu. 3. Ja redzamība ir publiska, visiem lietotājiem tiek dota pieeja receptei. 4. Ja redzamība ir privāta, tiek salīdzināts receptes autora identifikators ar lietotāju identifikatoru. 5. Ja tie sakrīt recepte ir pieejama lietotājam. 6. Ja redzamība ir mājsaimniecības ietvaros, tiek nolasīts receptes autora un lietotāja mājsaimniecības identifikators. 7. Ja tie sakrīt recepte ir pieejama lietotājam. 8. Citādi recepte nav pieejama lietotājam.
Izvadīti
1. Pieeja receptes informācijai.
Kļūdas paziņojumi
1. Nav piekļuves tiesību attiecīgajai receptei.
Izsaucamās prasības
1. Pieejama reģistrētiem lietotājiem.

6. Jauno lietotāju konta pievienošana:
 - 6.1. Jānodrošina, ka jebkurš lietotājs (tai skaitā viesi) var izveidot sev jaunu lietotāju kontu.
 - 6.2. Jāļauj izveidot jaunu lietotāju kontu, izmantojot formu, kur tiek norādīti nepieciešamie personas dati, aprakstīti ieejas informācijas apraksta 1.punktā.
 - 6.3. Ja obligātie lauki nav izpildīti, tad neļauj izveidot jaunu lietotāju kontu un tiek izvadīts paziņojums.
 - 6.4. Ja informācija par lietotāju ar ievadīto e-pastu pastāv jau sistēmā, neļaut pievienot kontu un izvadīt ziņojumu.
7. Lietotāju kontu rediģēšana un dzēšana:
 - 7.1. Jānodrošina, ka lietotājs, kam pieder lietotāja konts var tikai rediģēt un dzēst savu profilu.
 - 7.2. Rediģēt un dzēst informāciju par jebkuru lietotāju var tikai administrators.
 - 7.3. Lietotāju profilu rediģēšanai ir jāpiedāvā forma, kur jāievada jaunie personas dati.
 - 7.4. Ja formāts neatbilst ieejas informācijas apraksta 1. punktā minētajām, neļaut pievienot informāciju un izvadīt ziņojumu.
 - 7.5. Pirms jaunās informācijas profila dzēšanas jāizvada brīdinājuma ziņojums ar iespēju saglabāt vai atcelt veiktās izmaiņas.

- 7.6. Pirms konta dzēšanas no sistēmas tiek prasīts apstiprinājums no lietotāja par konta dzēšanu.
8. Mājsaimniecības un tās informācijas pārvaldīšana:
 - 8.1. Jānodrošina, ka jebkurš lietotājs var pieslēgties pie cita lietotāja izveidotas mājsaimniecības vai izveidot jaunu mājsaimniecību (sīkāk aprakstīts funkcionālo prasību 2.punktā).
 - 8.2. Jānodrošina, ka mājsaimniecības informāciju un tās rediģēšanu un dzēšanu var pārvaldīt tikai tās īpašnieks.
 - 8.3. Mājsaimniecības rediģēšanai jānodrošina forma, kur tiek norādīti ieejas informācijas 2.punkta norādītie lauki un citi lietotāji mājsaimniecībā ar iespēju noņemt lietotājus no mājsaimniecības.
 - 8.4. Ja formāts neatbilst, neļaut rediģēt informāciju par mājsaimniecību un tiek izvadīts paziņojums.
 - 8.5. Pirms mājsaimniecības dzēšanas tiek izvadīts brīdinājuma ziņojumus lietotājam.
 - 8.6. Ja tiek izdzēsta mājsaimniecība, tad visi mājsaimniecības produkti saistīti ar attiecīgo mājsaimniecību tiek dzēsti no sistēmas.
9. Mājsaimniecības pārtikas produktu un to informācijas pārvaldīšana
 - 9.1. Jānodrošina, ka jebkurš mājsaimniecības lietotājs var pievienot (sīkāk apraksts funkcionālo prasību 3.punktā), rediģēt un noņemt pārtikas produktu attiecīgajai mājsaimniecībai.
 - 9.2. Jāļauj rediģēt datus par jau esošu pārtikas produktu mājsaimniecībai, izmantojot formu.
 - 9.3. Ja formāts neatbilst 3.punkta un 4.punkta aprakstītajam, izvadīt paziņojumu un netiek mainīti ieraksta dati.
 - 9.4. Pirms mājsaimniecības produkta dzēšanas tiek izvadīts brīdinājuma ziņojumus lietotājam par pārtikas produkta dzēšanu no mājsaimniecības.
10. Receptes un tās informācijas pārvaldīšana:
 - 10.1. Jānodrošina, ka jebkurš lietotājs var izveidot recepti.
 - 10.2. Jāļauj izveidot recepti, izmantojot formu, kur ieejas dati atbilst ieejas informācijas apraksta 3., 6., 7., 8. un 9.punktam.
 - 10.3. Jānodrošina receptes redzamība atbilstoši lietotāja norādījumam (privāta, mājsaimniecības ietvaros, publiska).

- 10.4. Ja obligātie lauki nav aizpildīti, tad neļauj izveidot recepti un tiek izvadīts paziņojums un recepte netiek izveidota.
- 10.5. Jānodrošina, ka lietotājs, kas izveidoja recepti var rediģēt un dzēst tikai savas receptes.
- 10.6. Dzēst informāciju par jebkuru recepti var tikai administrators.
- 10.7. Receptes rediģēšanai ir jāpiedāvā forma, kur jāievada jaunie dati.
- 10.8. Ja formāts neatbilst ieejas informācijas apraksta 3., 6., 7., 8. un 9.punktam minētajām, neļaut pievienot informāciju un izvadīt ziņojumu.
- 10.9. Pirms receptes dzēšanas no sistēmas tiek prasīts apstiprinājums no lietotāja par receptes dzēšanu.
- 10.10. Ja tiek izdzēsta recepte, tad tā tiek dzēsta arī no visu lietotāju izveidotajām mapēm. Visas atsauksmes, kas bija zem receptes arī tiek dzēstas no sistēmas.
11. Atsauksmju un to informācijas pārvaldīšana:
 - 11.1. Jānodrošina, ka jebkurš lietotājs receptei var izveidot atsauksmi.
 - 11.2. Jāļauj izveidot atsauksmi, izmantojot formu, kur ievadītie dati atbilst ieejas informācijas 11.punktam.
 - 11.3. Ja obligātie lauki nav aizpildīti, tad neļauj izveidot atsauksmi un tiek izvadīts paziņojums un atsauksme netiek izveidota.
 - 11.4. Jānodrošina, ka lietotājs, kas izveidoja atsauksmi var rediģēt un dzēst tikai savas atsauksmes.
 - 11.5. Dzēst informāciju par jebkuru atsauksmi var tikai administrators.
 - 11.6. Atsauksmes rediģēšanai ir jāpiedāvā forma, kur jāievada jaunie atsauksmes dati.
 - 11.7. Ja formāts neatbilst ieejas informācijas apraksta 11. punktā minētajām, neļaut pievienot informāciju un izvadīt ziņojumu.
 - 11.8. Pirms atsauksmes dzēšanas no sistēmas tiek prasīts apstiprinājuma ziņojums no lietotāja par atsauksmes dzēšanu.
12. Iepirkšanās saraksta un tā informācijas pārvaldīšana:
 - 12.1. Jānodrošina, ka katrai mājsaimniecībai ir savs iepirkšanās saraksts.
 - 12.2. Lietotājam jāļauj pievienot jaunu pārtikas produktu ierakstus, izveidojot formu, kur ievadītie dati atbilst ieejas informācijas 12. punktam.
 - 12.3. Ja obligātie lauki nav aizpildīti un formāts nav atbilstošs, neļaut pievienot pārtikas produktu un izvadīt atbilstošu ziņojumu.

- 12.4. Jānodrošina, ka lietotāji no atbilstošās mājsaimniecības iepirkšanās sarakstā var rediģēt un dzēst pārtikas produktus.
- 12.5. Iepirkšanās saraksta rediģēšanai ir jāpiedāvā forma, kur jāievada jaunie pārtikas produkta dati.
- 12.6. Ja formāts neatbilst ieejas informācijas apraksta 12. punktā minētajām, neļaut pievienot informāciju un izvadīt ziņojumu.
- 12.7. Pirms pārtikas produkta dzēšanas jāizvada brīdinājuma ziņojums ar iespēju apstiprināt vai atcelt darbību.
13. Mapes un tās informācijas pārvaldīšana:
 - 13.1. Jānodrošina, ka jebkurš lietotājs var izveidot mapi.
 - 13.2. Jāļauj izveidot mapi, izveidojot formu, kur ievadītie dati atbilst ieejas informācijas 10.punktam.
 - 13.3. Ja obligātie lauki nav izpildīti, tad neļauj izveidot mapi un tiek izvadīts paziņojums un mape netiek izveidota.
 - 13.4. Jānodrošina, ka lietotājs, kas izveidoja mapi var rediģēt un dzēst tikai savas mapes.
 - 13.5. Mapes rediģēšanai ir jāpiedāvā forma, kur jāievada jaunie atsauksmes dati.
 - 13.6. Ja formāts neatbilst ieejas informācijas apraksta 10. punktā minētajām, neļaut pievienot informāciju un izvadīt ziņojumu.
 - 13.7. Jānodrošina, ka lietotājs var pievienot vai noņemt receptes savās izveidotajā mapēs.
 - 13.8. Ja atteicīgā mape neeksistē, tad tiek izvadīts paziņojums un recepte netiek pievienota mapei.
 - 13.9. Pirms mapes dzēšanas no sistēmas tiek prasīts apstiprinājuma ziņojums no lietotāja par mapes dzēšanu.
14. Pārtikas produktu un to informācijas pārvaldīšana:
 - 14.1. Jānodrošina, ka tikai administrators vai izveidot produktu un ar to saistītu informāciju.
 - 14.2. Jāļauj izveidot pārtikas produktu kategoriju, izmantojot formu, kur ievadītie dati atbilst ieejas informācijas 5.punktam.
 - 14.3. Jāļauj izveidot pārtikas produktu, izmantojot formu, kur ievadītie dati atbilst ieejas informācijas 3.punktam.
 - 14.4. Ja obligātie lauki nav izpildīti, tad neļauj izveidot pārtikas produktu vai pārtikas kategoriju un tiek izvadīts paziņojums un tie netiek izveidoti.

- 14.5. Jānodrošina, ka administrators var rediģēt un dzēst pārtikas produktus un pārtikas produktu kategorijas.
- 14.6. Pārtikas produkta vai pārtikas produkta kategorijas rediģēšanai ir jāpiedāvā forma, kur jāievada jaunie dati.
- 14.7. Ja formāts neatbilst ieejas informācijas apraksta 3. punktā un 5.punktā minētajām, neļaut pievienot informāciju un izvadīt ziņojumu.
- 14.8. Pirms pārtikas produkta vai tā kateogrijas dzēšanas no sistēmas tiek prasīts apstiprinājuma ziņojums no lietotāja par atsauksmes dzēšanu.
- 15. Receptu klasificēšanas informācijas pārvaldīšana:
 - 15.1. Jānodrošina, ka tikai administrators vai izveidot receptes kategorijas un receptes tipus.
 - 15.2. Jāļauj izveidot receptes kategoriju, izmantojot formu, kur ievadītie dati atbilst ieejas informācijas 6.punktam.
 - 15.3. Jāļauj izveidot receptes tipu, izmantojot formu, kur ievadītie dati atbilst ieejas informācijas 7.punktam.
 - 15.4. Ja obligātie lauki nav izpildīti, tad neļauj izveidot receptes tipu vai receptes kategoriju un tiek izvadīts paziņojums un tie netiek izveidoti.
 - 15.5. Jānodrošina, ka administrators var rediģēt un dzēst receptes kateogrijas un receptes tipus.
 - 15.6. Receptes kategorijas vai receptes tipa rediģēšanai ir jāpiedāvā forma, kur jāievada jaunie dati.
 - 15.7. Ja formāts neatbilst ieejas informācijas apraksta 3. punktā un 5.punktā minētajām, neļaut pievienot informāciju un izvadīt ziņojumu.
 - 15.8. Pirms receptes tipa vai receptes kateogrijas dzēšanas no sistēmas tiek prasīts apstiprinājuma ziņojums par atsauksmes dzēšanu.
- 16. Pārtikas produktu uzskaitē pēc receptes izpildes:
 - 16.1. Jānodrošina iespēja lietotājam atzīmēt recepti kā pabeigtu.
 - 16.2. Pēc receptes atzīmēšanas kā pabeigtas jāparāda paziņojums par izmantoto pārtikas produktu atņemšanu no lietotāja mājāsaimniecības.
 - 16.3. Pirms pārtikas produktu noņemšanas jāizvada brīdinājuma ziņojums ar iespēju apstiprināt vai atcelt darbību.
- 17. Receptes izdruka:
 - 17.1. Lietotājam jābūt iespējai apskatīt un ģenerēt izvēlētās receptes PDF izdruku.

- 17.2. Jānodrošina, ka sistēmai izvada datus par recepti, tostarp receptes autoru, lietotāju novērtējumu, gatavošanas laikus, pārtikas produktiem to daudzumu, pagatavošanas soļiem.
- 17.3. Jānodrošina PDF dokumenta druku un lejuplādi.
- 18. E-pasta paziņojums par pārtikas produktu derīguma termiņu tuvošanos.
- 18.1. Jānodrošina, ka sistēma aprēķina dienu skaitu līdz lietotāja mājāsaimniecības pārtikas produkta derīguma termiņa datumam.
- 18.2. Ja ir atlikušas tikai dažas dienas vai derīguma termiņš jau ir sasniegts, tiek nosūtīts brīdinājums uz lietotāja e-pastu.

2.3. Nefunkcionālās prasības

- 1. Zemāk aprakstītas prasības par sistēmas lietojamību un dizainu.
 - 1.1. Sistēmas saskarnes valodai ir jābūt latviešu valodai.
 - 1.2. Sistēmai jādarbojas visos populārākajos pārlūkos, tostarp Google Chrome, Mozilla Firefox, Microsoft Edge un Safari.
 - 1.3. Dizainam ir jābūt vienkāršam, intuitīvam, bēšos un sarkanos toņos.
 - 1.4. Tīmekļa lietojumprogrammai jābūt responsīvai, dizainam pielāgojoties dažādām ierīcēm.
 - 1.5. Sistēmai jāreaģē uz lietotāja darbībām bez ievērojamas aiztures, nodrošinot ērtu un neapgrūtinātu lietošanu.
 - 1.6. Lietotāja reģistrēšanas formai jāizskatās sekojoši (sk. 2.1.att.).

Reģistrēšanās

Vārds

E-pasts

Lietotājvārds

Parole

Parole atkārtoti

Reģistrēties

[Ir lietotāja kods?](#)

2.1.attēls Reģistrācijas veidlapas skice

1.7. Lietotāja pieslēgšanās formai jāizskatās sekojoši (sk. 2.2.att.).

Pieslēgšanās

E-pasts

Parole

Pieslēgties

[Nav lietotāja konta?](#)

2.2.attēls Pieslēgšanās veidlapas skice

1.8. Dzēšanas brīdinājuma paziņojumam jāizskatās sekojoši (sk. 2.3.att.).

Dzēšana

Vai tiešām vēlaties šo vienumu dzēst?

Atcelt Apstiprināt

2.3.att. Dzēšanas brīdinājuma paziņojuma skice

1.9. Receptes vienumam jāizskatās sekojoši (sk. 2.4.att.).

Receptes nosaukums ★4.1

12min Nav 2 sastāvdaļu

2.4.att. Receptes vienuma skice

2. Zemāk aprakstītas prasības par sistēmas drošību.

- 2.1. Sistēmas izstrādē jāievēro OWASP [7] vadlīnijas, lai novērstu populārākās un aktuālākās tīmekļa lietojumprogrammu ievainojamības.
- 2.2. Sensitīvie dati (piem. lietotāju paroles, autentifikācijas dati) jāglabā hešotā veidā.
- 2.3. Sistēmai jānodrošina lietotāju autentifikācija, autorizācija un lomu pārvaldība, ierobežojot piekļuvi atbilstoši lietotāja lomām.
- 3. Zemāk aprakstīta veikspēja un pieejamība.
 - 3.1. Sistēmai jānodrošina lapu ielāde ne ilgāk kā 3 sekundes pie normālas slodzes.
 - 3.2. Sistēmai jānodrošina darbība arī atsevišķu moduļu kļūmes gadījumos, saglabājot pamata funkcionalitāti.

3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS

Izstrādājamā sistēma ir tīmekļa lietojumprogramma, kas paredzēta mājsaimniecības pārtikas krājumu uzskaitēi, produktu derīguma termiņu kontrolei un recepšu ieteikumu izveidei, balstoties uz pieejamajiem produktiem. Sistēma sastāv no divām galvenajām daļām: lietotāja puses (frontend), kas nodrošina lietotāja saskarni un mijiedarbību, un servera puses (backend), kas veic datu apstrādi un uzglabāšanu.

Lietotāja puses izstrādei tiek izmantots **JavaScript (ES6/ES2015+)** kopā ar ietvaru **Vue.js 3.5**. [9], kas nodrošina komponentēs balstītu arhitektūru un dinamisku lietotāju saskarni. Vue.js izvēlēts tā vienkāršības, elastības un veiktsējas dēļ. Lietotāja saskarnes struktūras veidošanai tiek izmantots **HTML5**, bet vizuālajam noformējumam **CSS3**, izmantojot **Sass (SCSS) 1.94** [8].

Lai nodrošinātu efektīvu saziņu starp frontend un backend bez tradicionālas REST API izmantošanas, tiek izmantots **Inertia.js 2.1** [3], kas ļauj veidot vienlapas lietojumprogrammu (SPA), izmantojot servera puses maršrutēšanu. Papildus tam tiek izmantots **Ziggy.js 2.6.**, kas ļauj izmantot Laravel definētos maršrutus JavaScript vidē, tādējādi vienkāršojot navigācijas loģiku.

Servera daļas izstrādē sistēmā tiek izmantota programmēšanas valoda **PHP 8.3**, kas nodrošina augstu veiktsēji un drošību. Kā galvenais backend ietvars tiek izmantots **Laravel 12.x** [4], kas piedāvā plašu funkcionalitāti, tostarp maršrutēšanu, ORM (Object-Relational Mapping), validāciju, kešošanu un drošības mehānismus. Lietotāju autentifikācijas nodrošināšanai tiek izmantots **Laravel Fortify 1.32.**, kas nodrošina gatavu funkcionalitāti lietotāju reģistrācijai, autorizācijai, paroli atjaunošanai un divu faktoru autentifikācijai.

Datu uzglabāšanai tiek izmantota relāciju datubāzu pārvaldības sistēma **MySQL 8.3**, kas nodrošina uzticamu un strukturētu datu glabāšanu.

Administrācijas paneļa izstrādei tiek izmantots **Laravel Filament 3**. [2], kas nodrošina gatavas CRUD (Create, Read, Update, Delete) funkcionalitātes, datu tabulas, filtrus un vizuālas formas. Papildus tiek izmantoti vairāki **Cube Aģentūras** [1] izstrādāti moduļi, piemēram, *filament-page-manager* (template-bāzēta lapu pārvaldība), kas ļauj izveidot un strukturēt lapas, izmantojot konfigurējamus šablonus Filament panelī, *filament-translations* tulkojumu pārvaldībai, *filament-redirects* URL pāradresāciju pārvaldībai un *filament-constructor* lapu un satura veidošanai. Šie moduļi paplašina Filament

funkcionalitātes ar satura pārvaldības funkcionalitāti, saglabājot konsekventu paneles struktūru un vienkāršojot administrācijas darbplūsmu.

E-pasta funkcionalitātes testēšanai izstrādes laikā tiek izmantots **Mailtrap** [5], kas ļauj droši testēt e-pastu nosūtīšanu bez reālas piegādes lietotājiem, kā arī nodrošina iespēju produkcijas vidē sūtīt īstus e-pastus.

PDF izdrukas funkcionalitātes nodrošināšanai tiek izmantots **mPDF** [6], kas ļauj vienkārši un efektīvi veidot PDF izdrukas gan no HTML failiem, gan saitēm.

Lokālās izstrādes vides nodrošināšanai tiek izmantots **Docker Desktop** kopā ar **Laravel Sail**, kas ļauj ērti izveidot un pārvaldīt konteinerizētu izstrādes vidi bez nepieciešamības manuāli konfigurēt serverus un atkarības. Šāda pieeja nodrošina vienotu un atkārtojamu vidi visos izstrādes posmos, kā arī vienkāršo projekta uzstādīšanu uz dažādām ierīcēm.

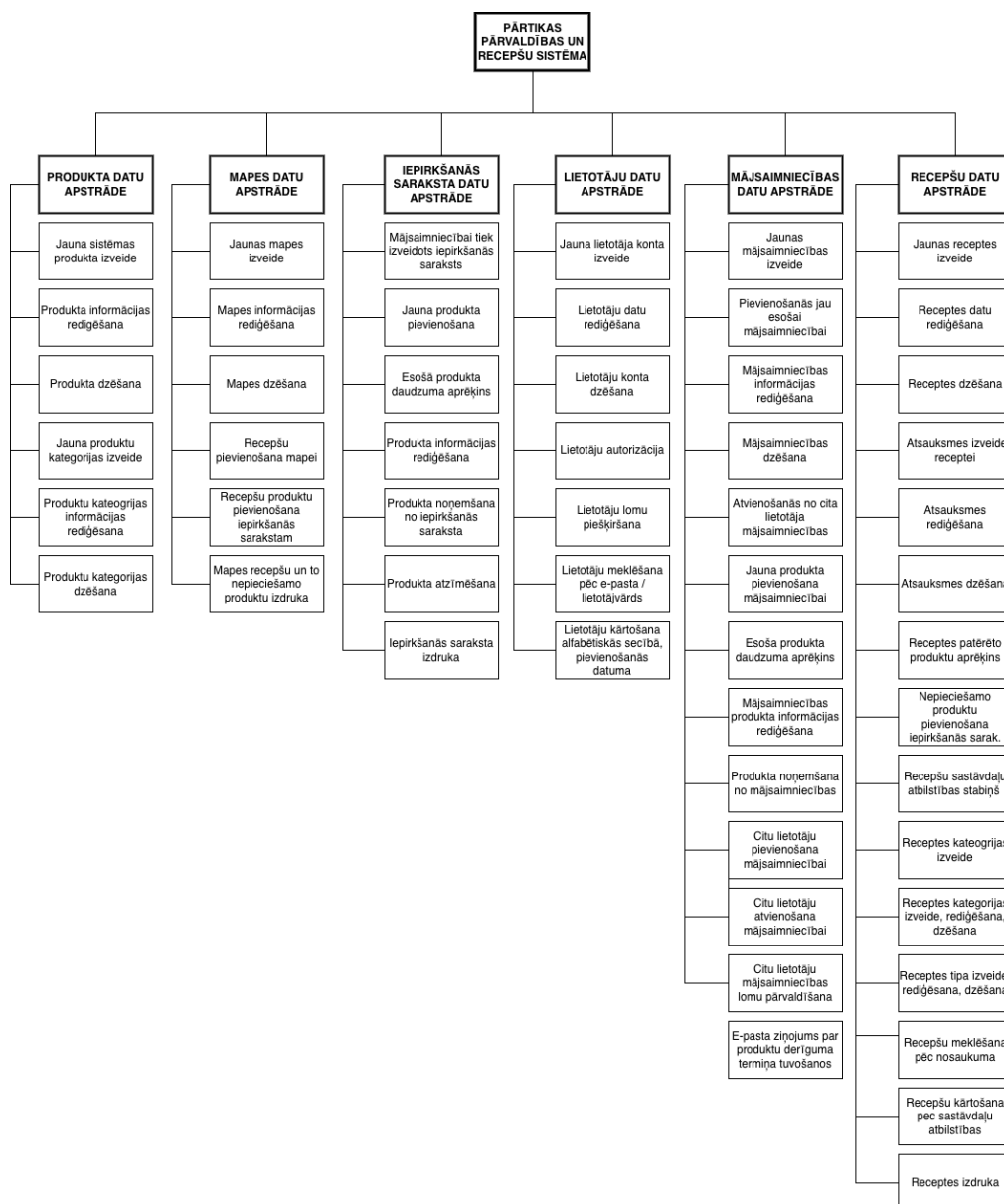
Koda izstrādei tiek izmantota **JetBrains PhpStorm** izstrādes vide, kas nodrošina sintakses pārbaudi, automātisko koda papildināšanu, atklūdošanas iespējas un integrāciju ar versiju kontroles sistēmu **Git**.

4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA

4.1. Sistēmas struktūras modelis

4.1.1. Sistēmas arhitektūra

Sistēma sastāvēs no 6 apakšsistēmām (sk. 4.1. att.): produktu datu apstrādes apakšsistēmas, mapes datu apstrādes apakšsistēmas, iepirkšanās saraksta datu apstrādes apakšsistēmas, lietotāju datu apstrādes apakšsistēmas, mājsaimniecības datu apstrādes apakšsistēmas un recepšu datu apstrādes apakšsistēmas.



4.1.att. Funkcionālās dekompozīcijas hierarhiskā diagramma

Zemāk aprakstītās apakšsistēmas tika iedalītas pēc sistēmas darbības sfērām.

- **Produktu datu apstrādes** 6 modeļi nodrošina administratoram pārvaldīt informāciju par sistēmas produktiem, kurus lietotājiem ir iespēja pievienot savās mājsaimniecībās, iepirkšanās sarakstos un receptēs.
- **Mapes datu apstrādes** 6 modeļi nodrošina lietotājiem iespēju pārvaldīt recepšu sarakstus un to informāciju, dodot iespēju pievienot sarakstu receptēs trūkstošos produktus iepirkšanās sarakstam.
- **Iepirkšanās sarakstu datu apstrādes** 7 modeļi nodrošina katrai mājsaimniecībai iepirkšanās sarakstu, kur lietotājiem ir iespēja pārvaldīt datus par produktiem tajā.
- **Lietotāju datu apstrādes** 7 modeļi nodrošina administratoram pārvaldīt lietotāju lomas un ļaut lietotājiem izveidot un pārvaldīt savus lietotāju kontus.
- **Mājsaimniecības datu apstrādes** 13 modeļi nodrošina lietotājam iespēju izveidot vai pievienoties citai mājsaimniecībai, pārvaldīt savas mājsaimniecības datus un produktus tajā.
- **Receptes datu apstrādes** 15 modeļi nodrošina administratoram iespēju pārvaldīt informāciju par receptēm, tās atsauksmēm, kategorijām un tiem. Lietotājiem ir iespēja pārvaldīt datus par savām receptēm un atsauksmēm.

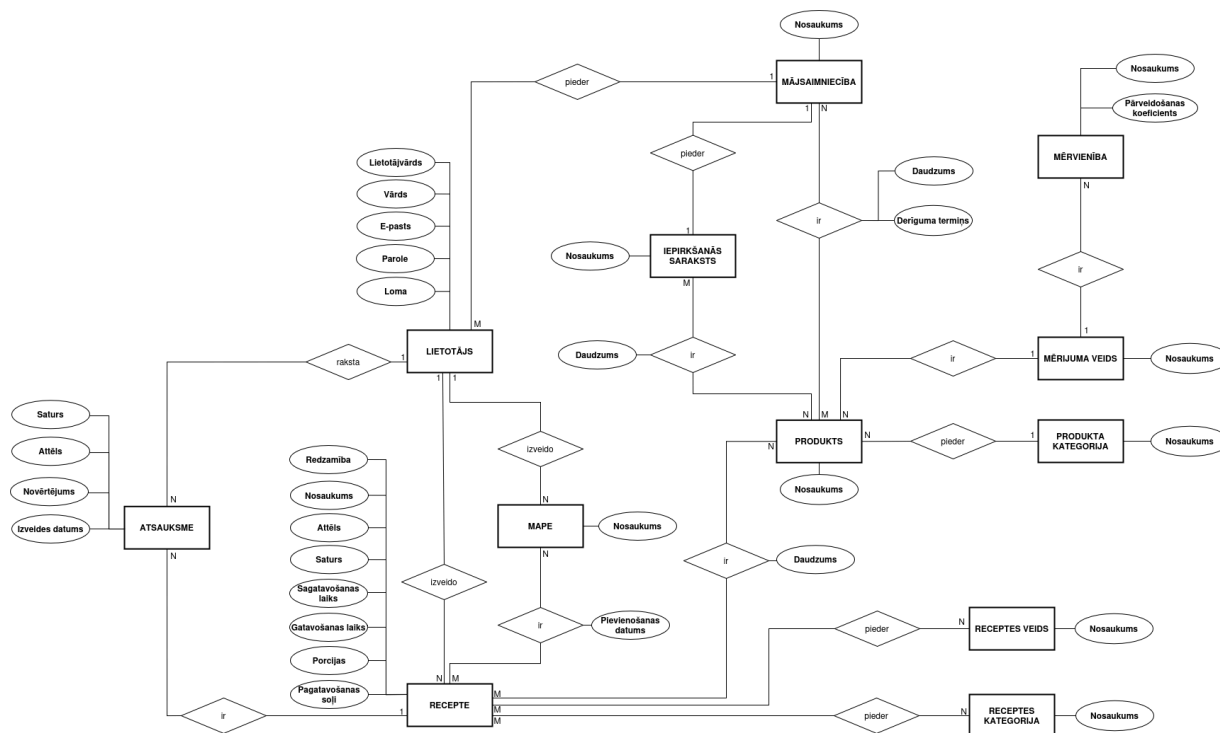
4.1.2. Sistēmas ER modelis

Datubāzes projektēšanā datu un saišu attēlošanai tika izmantota ER diagramma, kas sastāv no entītijām un relācijām. Projekta ER datu modelis (sk. 4.1.att.) sastāv no 10 entītijām, lai labāk izprastu kā mijiedarbojas šīs entītijas, tiek izmantotas relācijas. Zemāk aprakstītas ER datu modelī attēlotās relācijas.

- Starp entītijām **lietotājs** un **recepte** ir attiecība **viens pret daudziem**, jo viens lietotājs var izveidot vairākas receptes, bet viena recepte var izveidot tikai viens lietotājs.
- Starp entītijām **lietotājs** un **mape** ir attiecība **viens pret daudziem**, jo viens lietotājs var izveidot vairākas mapes, bet viena mape var piederēt tikai vienam lietotājam.
- Starp entītijām **mape** un **recepte** ir attiecība **daudzi pret daudziem**, jo vienā mapē var būt vairākas receptes, un viena recepte var būt vairākās mapēs.

- Starp entītijām **lietotājs** un **atsauksme** ir attiecība **viens pret daudziem**, jo viens lietotājs var rakstīt vairākas atsauksmes, bet vienu atsauksmi var rakstīt tikai viens lietotājs.
- Starp entītijām **recepte** un **atsauksme** ir attiecība **viens pret daudziem**, jo vienai receptei var būt vairākas atsauksmes, bet viens atsauksme var būt tikai pie vienas receptes.
- Starp entītijām **mājsaimniecību** un **lietotājs** ir attiecība **viens pret daudziem**, jo viena mājsaimniecība var piederēt vairākiem lietotājiem, bet viens lietotājs var būt piesaistīts tikai vienai mājsaimniecībai.
- Starp entītijām **mājsaimniecība** un **produkts** ir attiecība **daudzi pret daudziem**, jo vienā mājsaimniecībā ir vairāki produkti un viens produkts var būt vairākās mājsaimniecībās.
- Starp entītijām **mājsaimniecību** un **iepirkumu saraksts** ir attiecība **viens pret viens**, jo vienai mājsaimniecībai var būt tikai viens iepirkumu saraksts, un viens iepirkumu saraksts var piederēt tikai vienai mājsaimniecībai.
- Starp entītijām **iepirkumu saraksts** un **produkts** ir attiecība **daudzi pret daudziem**, jo vienā iepirkumu sarakstā var būt vairāki produkti, un viens produkts var būt vairākos iepirkumu sarakstos.
- Starp entītijām **produkta kategorija** un **produkts** ir attiecība **viens pret daudziem**, jo vienai kategorijā ir vairāki produkti, bet viens produkts var būt tikai vienā kategorijā.
- Starp entītijām **mērvienības veids** un **produkts** ir attiecība **viens pret daudziem**, jo viens mērvienības veids var būt vairākiem produktiem, bet vienam produktam var būt tikai viens mērvienības veids.
- Starp entītijām **mērvienības veids** un **mērvienība** ir attiecība **viens pret daudziem**, jo vienam mērvienības veidam ir vairākas mērvienības, bet vienai mērvienībai ir tikai viens mērvienības veids.
- Starp entītijām **recepte** un **produkts** ir attiecība **daudzi pret daudziem**, jo vienai receptei ir vairāki produkti un viens produkts var būt vairākās receptēs.
- Starp entītijām **recepte** un **receptes kategorija** ir attiecība **daudzi pret daudziem**, jo vienā receptes kategorijā var būt vairākas receptes, un viena recepte var piederēt vairākas receptes kategorijas.

- Starp entitijām **recepte** un **receptes veids** ir attiecība daudzi pret daudziem, jo pie viena receptes tipa var piederēt vairākas receptes, un vienai receptei var piederēt vairāki receptes tipi.



4.2.att. Sistēmas datu konceptuālais modelis

4.2. Funkcionālais sistēmas modelis

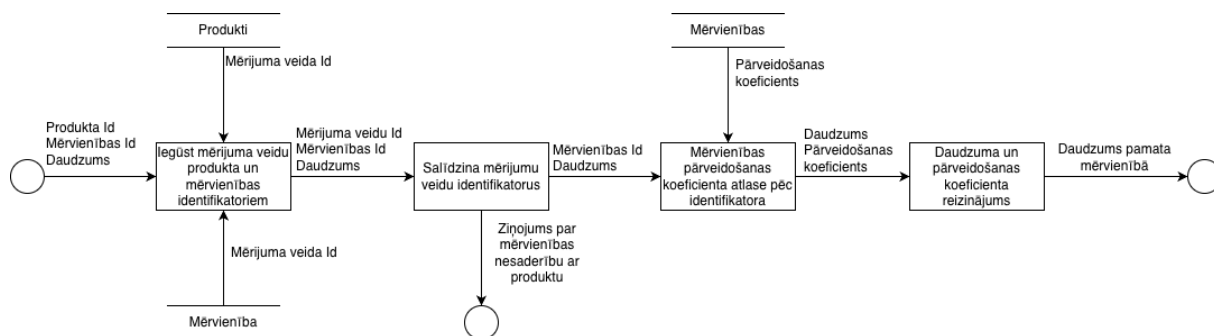
4.2.1. Datu plūsmu modelis

Šajā sadaļā tiek aprakstīti svarīgākie procesi no iepriekš aprakstītajiem sistēmas arhitektūras attēlotajām funkcijām (sk. 4.1.att.), kuri ir būtiski izstrādāti sistēmas programmas atspoguļojums.

1. Produkta daudzuma pārveidošana uz pamata mērvienību (sk. 4.3.att.).

Saskarne padod produkta un mērvienības identifikatorus un produkta daudzumu. Tiek iegūti produkta un mērvienības mērvienības veidi (tilpuma, svara vai skaita) un salīdzināti. Ja tie nesakrīt, tad tiek izvadīts ziņojums par mērvienības un produkta nesaderību un daudzums netiek pārveidots. Citādāk no mērvienību tabulas tiek iegūts atbilstošais pārveidošanas koeficients attiecīgajai mērvienībai un aprēķināts daudzums pamata veida mērvienībā (grami, mililitri un gabali), kas tiek padods atpakaļ sistēmai. Datubāze visi daudzumi tiek glabāti

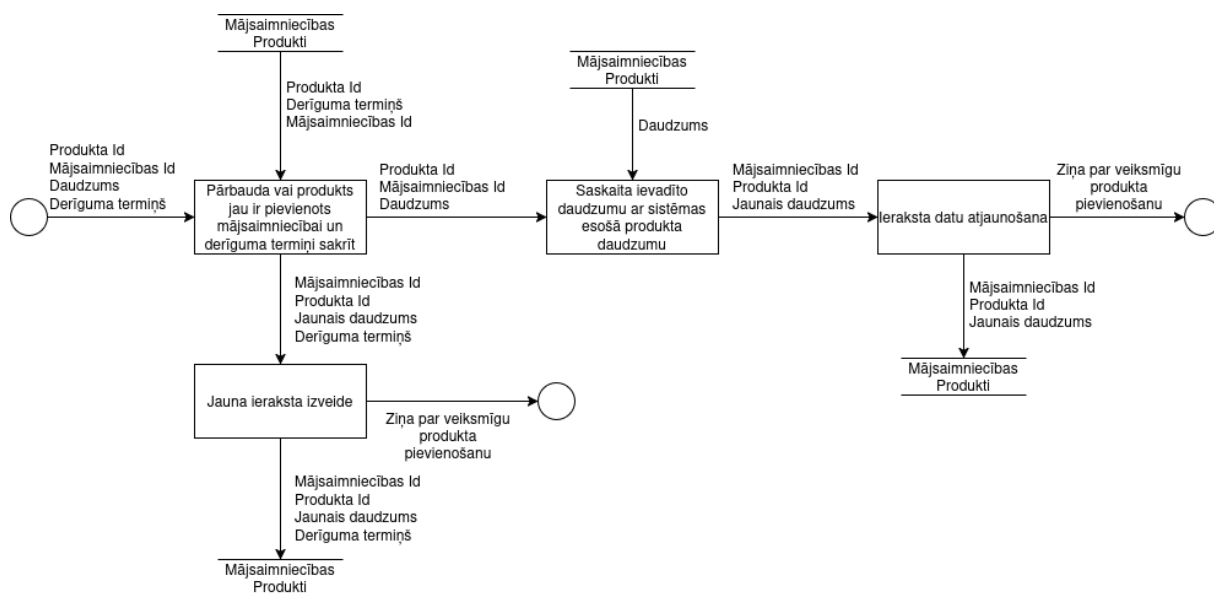
pamata mērvienībās. Modulis tiek pielietots katru reizi, kad sistēmā notiek aprēķini ar produktu daudzumiem.



4.3.att. Daudzuma pārveidošanas datu plūsmas diagramma

2. Produkta pievienošana mājsaimniecībā (sk. 4.4.att.).

Saskarne padod produkta un mājsaimniecības identifikatoru, produkta daudzumu, produkta derīguma termiņu. Tiek salīdzināti produkta un mājsaimniecības identifikatori un derīguma termiņš ar saskarnes ievadi, ja tie nesakrīt, tad tiek izveidots jauns ieraksts mājsaimniecību produktos un izvadīts ziņojums lietotājam par veiksmīgu produkta pievienošanu. Citādāk tiek iegūta esošā produkta un ievadītā produkta daudzuma summa, atjaunojot mājsaimniecības produktu daudzuma datus un izvadot paziņojumu par veiksmīgu produkta pievienošanu lietotājam.

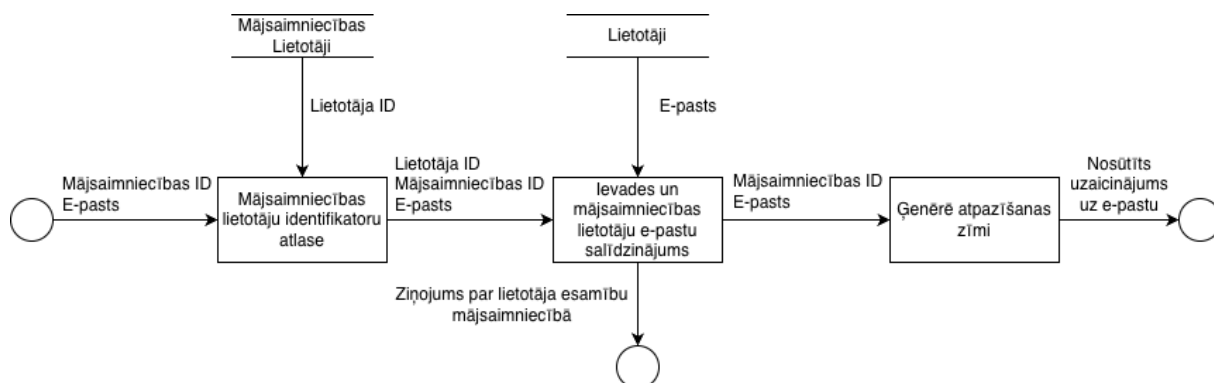


4.4.att. Produkta mājsaimniecības pievienošana datu plūsmas diagramma

3. Lietotāja pievienošanās mājsaimniecībai, uzaicinājuma nosūtīšana (sk. 4.5.att.).

Saskarne padod mājsaimniecības identifikatoru un vēlamu e-pastu uzaicinājumam. Tiek atlasīti mājsaimniecībā jau esošie lietotāji un to e-pasti salīdzināti, ja mājsaimniecībā jau

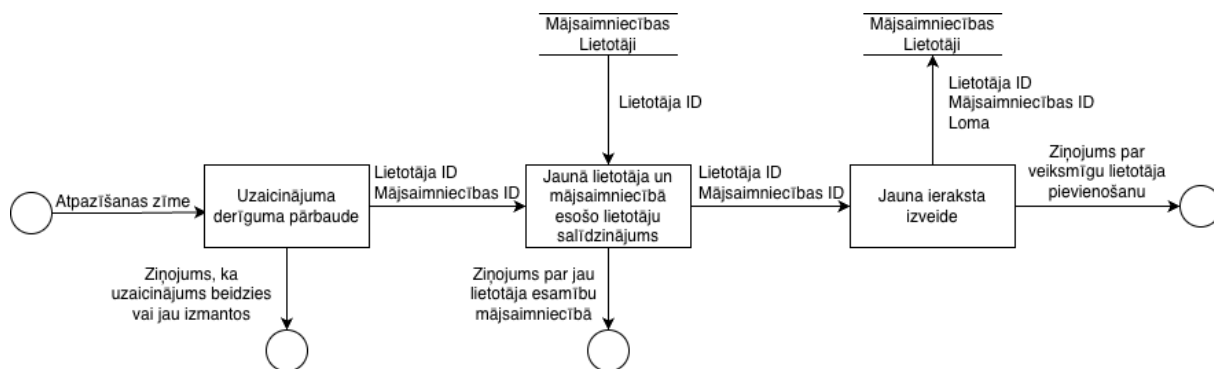
ir lietotājs ar tādu e-pastu tiek izvadīts paziņojums un izacinājums netiek nosūtīts. Citādāk tiek padots mājsaimniecības identifikators ar e-pasta adresi un ģenerēta atpazīšanas zīme, nodrošinot e-pasta sesiju. Tālāk uz ievadīto e-pastu tiek nosūtīts uzaicinājums.



4.5.att. Pievienošanās uzaicinājuma nosūtīšanas datu plūsmas diagramma

4. Lietotāja pievienošanās mājsaimniecībai, uzaicinājuma akceptēšana (sk.4.6.att.).

Saskarne padod lietotāja uzaicinājuma atpazīšanas zīmi. Tiek pārbaudīta vai sesija vēl ir derīga - uzaicinājuma datums ir pagājis vai jau saite izmantota, tiek paziņots lietotājam, ka saite vairs nav derīga. Citādāk tiek padots lietotāja un mājsaimniecību identifikatori, lai salīdzinātu ar mājsaimniecībā esošajiem lietotājiem. Ja tie sakrīt tiek izvadīts ziņojums, ka lietotājs jau ir mājsaimniecībā un lietotājs netiek pievienots, ja sakrīt - tiek izveidots jauns ieraksts mājsaimniecības lietotāju tabulā ar lietotāja un mājsaimniecības identifikatoru un lomu (pēc noklusējuma biedrs), paziņojot par veiksmīgu darbību.

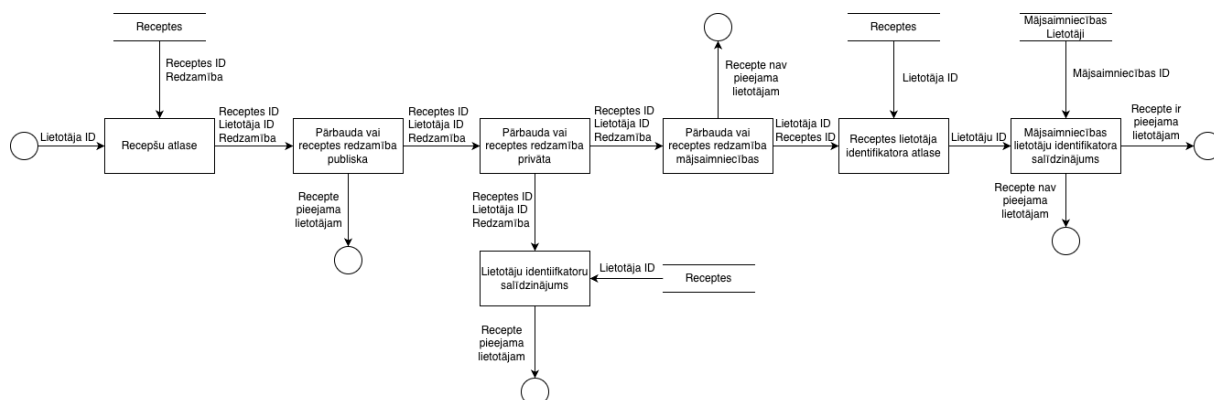


4.6.att. Pievienošanās mājsaimniecībai no uzaicinājuma datu plūsmas diagramma

5. Receptes redzamība lietotājam (sk. 4.7.att.).

Saskarne padod lietotāja identifikatoru, kā arī ielasa receptu identifikatorus ar to redzamību (privāta, publiska vai mājsaimniecības ietvaros). Tālāk tiek padota iegūtā informācija par receptēm un lietotāja identifikators. Tiek pārbaudīta vai receptes redzamība ir

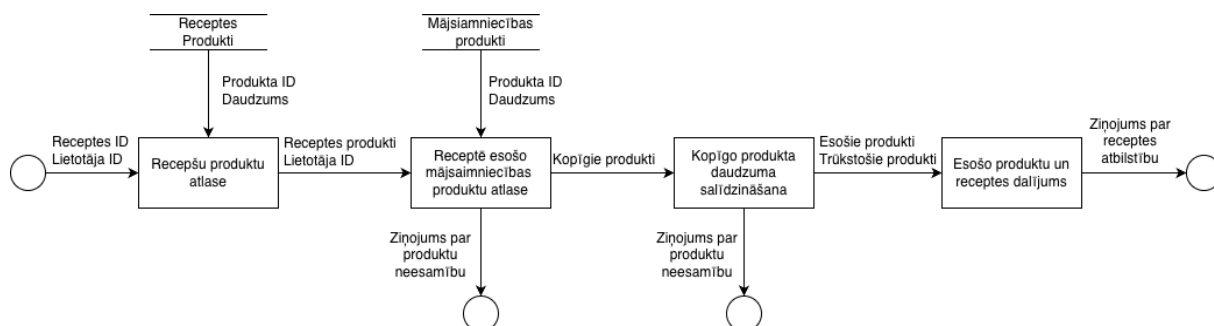
publiska, ja jā recepti lietotājam ir pieejama. Citādāk, ja receptes redzamība ir privāta, tad tiek iegūts receptes veidotāja identifikators un salīdzināts ar lietotāja. Gadījumā kad tie sakrīt recepte lietotājam ir pieejama. Pretējā gadījumā tālāk tiek pārbaudīts vai receptes redzamība ir mājsaimniecības ietvaros. Tālāk atkal tiek iegūts receptes veidotāja identifikators, kuram kopā ar lietotāja identifikatoru tiek atlasīts mājsaimniecības identifikators. Ja tie sakrīt, tad recepte ir pieejama lietotājam, citādāk tā nav un tiek izvadīts ziņojums.



4.7.att. Receptes redzamības datu plūsmas diagramma

6. Receptes atbilstība lietotāja mājsaimniecības produktiem (sk. 4.8.att.).

Saskarne padod receptes un lietotāja identifikatorus. Tiek atlasīti atbilstošie receptes produkti ar to daudzumu katrai receptei. Tālāk tiek atlasīti mājsaimniecības produktu identifikatoru, kas sakrīt ar iepriekš atlasītajiem receptes produktu identifikatoriem. Ja tie nesakrīt tiek izvadīts paziņojums par produktu neesamību, citādāk tiek padots tālāk kopīgais produkts. Kopīgo produktu daudzumi tiek salīdzināti - ja mājsaimniecībā ir mazāk produkta nekā receptē, tad tas tiek atzīmēts kā trūkstošs, citādi kā esošs. Esošais produktu skaits tiek dalīts ar receptes kopējo produktu daudzumu un atbilstība tiek paziņota saskarnei.

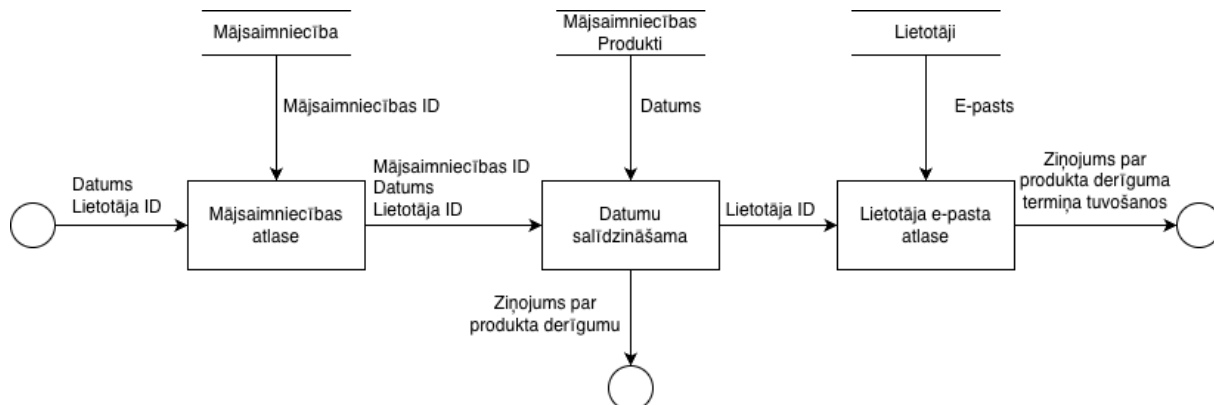


4.8.att. Receptes mājsaimniecības atbilstības aprēķina datu plūsmu diagramma

7. Derīguma termiņu tuvošanās atgādinājums (sk. 4.9.att.).

Saskarne padod pašreizējo datumu un lietotāja identifikatoru ar kura palīdzību tiek atlasīts mājsaimniecības identifikators. No mājsaimniecības produktu tabulas tiek atlasīti

produkta derīguma termiņa datumu un salīdzināti ar pašreizējo. Ja datumu starpība nav maza, tiek izvadīts ziņojums par produkta derīgumu, citādi tiek padots lietotāja identifikators e-pasta atlasei no lietotāju tabulas. Tiek nosūtīts e-pasts ar brīdinājumu par produkta derīguma termiņa tuvošanos.



4.9.att. Derīguma termiņa tuvošanās atgādinājums datu plūsmu diagramma

5. DATU STRUKTŪRAS APRAKSTS

Lai nodrošinātu datubāzes izveidi, ir nepieciešama datubāzes fiziskā struktūra. Tāpēc tika izveidota datubāzes tabulu saišu shēma (sk. 7.pielikumu). Datubāzes sastāv no 18 tabulām, kas satur informāciju par lietotājiem, atsauksmēm, mapēm, receptēm, recepšu kategorijām, recepšu veidiem, produktiem, produktu kategorijām, mērijuma veidiem, mērvienībām, iepirkšanās sarakstiem, mājsaimniecībām.

- Tabula “**users**” glabās informāciju par reģistrētiem lietotājiem.
- Tabula “**products**” glabās informāciju par pārtikas produktiem.
- Tabula “**product_category**” glabās informāciju par pārtikas produktu kategorijām (piem. Dārzeni).
- Tabula “**units**” glabās informāciju par pārtikas produktu mērvienībām.
- Tabula “**measurement_types**” glabās informāciju par mērijuma veidiem (piem. Svars, Tilpums, Skaitis).
- Tabula “**reviews**” glabās informāciju par lietotāju izveidotajām atsauksmēm attiecīgajai receptei.
- Tabula “**recipes**” glabās informāciju par receptēm.
- Tabula “**recipe_products**” glabā informāciju par receptē esošajiem pārtikas produktiem,
- Tabula “**recipe_categories**” glabās informāciju par receptes veidiem, kuras iedala pēc ēdiena veida (piem. Veģetārie ēdieni, Deserti).
- Tabula “**recipe_recipe_categories**” glabās informāciju par receptēm un tām piesaistītās receptes veidus.
- Tabula “**recipe_types**” glabās informāciju par receptes kategoriju, kuras iedala pēc ēdienreizes (piem. Brokastis).
- Tabula “**recipe_recipe_types**” glabās informāciju par receptēm un tās piesaistītajām receptes kategorijām.
- Tabula “**folders**” glabās informāciju par lietotāja izveidotajām mapēm.
- Tabula “**folder_recipes**” glabās informāciju par atteicīgās mapes esošajām receptēm.
- Tabula “**shopping_lists**” glabās informāciju par lietotāju iepirkšanās sarakstiem.

- Tabula “**shopping_list_products**” glabās informāciju par iepirkšanās saraksta esošajiem pārtikas produktiem.
- Tabula “**household**” glabās informāciju par lietotāja piesaistīto mājsaimniecību.
- Tabula “**household_products**” glabās informāciju par mājsaimniecībā esošajiem pārtikas produktiem.

Tabula “users” ir saistīta ar tabulām - “reviews”, “recipes”, “folders” - tabulu “households” caur lauku “household_id”. Tā satur informāciju (sk. 5.1. tabulu) kā identifikators, lietotājvārds, e-pasts, pilnais vārds, loma, parole.

5.1.tabula

Tabulas “**users**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	user_id	INT	-	Ieraksta identifikators
2.	username	VARCHAR	20	Lietotāja segvārds
3.	e-mail	VARCHAR	-	Lietotāja e-pasts
4.	name	VARCHAR	-	Lietotāja pilnais vārds
5.	role	VARCHAR	30	Lietotāja loma mājsaimniecībā
6.	password	VARCHAR	-	Lietotāja konta parole
7.	household_id	INT	-	Ārējā atslēga jeb identifikators tabulai “households”

Tabula “measurement_types” ir saistīta ar tabulu “units” un tabulu “products”. Tā satur informāciju (sk. 5.2. tabulu) kā identifikators un nosaukums.

5.2.tabula

Tabulas “**measurement_types**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	measurement_type_id	INT	-	Ieraksta identifikators
2.	name	VARCHAR	30	Mērijuma veida nosaukums

Tabula “units” ir saistīta ar tabulu “measurement_types” caur lauku “measurement_type_id”. Tā satur informāciju (sk. 5.3. tabulu) kā identifikators un nosaukums.

Tabulas “units” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	unit_id	INT	-	Ieraksta identifikators
2.	name	VARCHAR	10	Mērvienības nosaukums
3.	conversion_factor	DOUBLE	-	Mērvienības pārveidošanas koeficients uz mērvienības veida pamatvērtību
4.	measurement_type_id	INT	-	Ārējā atslēga jeb identifikators tabulai “measurement_types”

Tabula “products” ir saistīta ar tabulām - “household_products”, “shopping_list_products”, “recipe_products” - tabulu “product_categories” caur lauku “product_category_id” un tabulu “measurement_types” caur lauku “measurement_type_id”. Tā satur informāciju (sk. 5.4. tabulu) kā identifikators, nosaukums.

Tabulas “products” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	product_id	INT	-	Ieraksta identifikators
2.	name	VARCHAR	30	Pārtikas produkta nosaukums
3.	product_category_id	INT	-	Ārējā atslēga jeb identifikators tabulai “product_categories”
4.	measurement_type_id	INT	-	Ārējā atslēga jeb identifikators tabulai “measurement_types”

Tabula “product_categories” ir saistīta ar tabulu “products”. Tā satur informāciju (sk. 5.5. tabulu) kā identifikators, nosaukums.

Tabulas “product_categories” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	product_category_id	INT	-	Ieraksta identifikators
2.	name	VARCHAR	30	Pārtikas produkta kategorijas nosaukums

Tabula “reviews” ir saistīta ar tabulu “users” caur lauku “user_id” un ar tabulu “recipes” caur lauku “recipe_id”. Tā satur informāciju (sk. 5.6. tabulu) kā identifikators, saturs, attēla faila ceļš, novērtējums, izveides datums.

5.6.tabula

Tabulas “reviews” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	review_id	INT	-	Ieraksta identifikators
2.	content	VARCHAR	500	Atsauksme
3.	rating	INT	-	Novērtējums (1 līdz 5)
4.	created_at	DATETIME	-	Atsauksmes izveides datums
5.	user_id	INT	-	Ārējā atslēga jeb identifikators tabulai “users”
6.	recipe_id	INT	-	Ārējā atslēga jeb identifikators tabulai “recipes”

Tabula “recipes” ir saistīta ar tabulām - “reviews”, “recipe_recipe_types”, “recipe_recipe_categories”, “folder_recipes”, “recipe_products” - tabulu “users” caur lauku “user_id”. Tā satur informāciju (sk. 5.7. tabulu) kā identifikators, nosaukums, attēla ceļš, saturs, sagatavošanas laiks, gatavošanas laiks, porcijas, pagatavošanas soļi.

5.7.tabula

Tabulas “recipes” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	recipe_id	INT	-	Ieraksta identifikators
2.	visibility	ENUM	-	Receptes redzamība (privāts, mājsaimniecības ietvaros, publisks)
3.	name	VARCHAR	20	Receptes nosaukums
4.	image_src	VARCHAR	-	Receptes attēla faila ceļš
5.	prep_time	INT	-	Receptes sagatavošanās ilgums minūtēs
6.	cook_time	INT	30	Receptes gatavošanās ilgums minūtēs
7.	servings	INT	-	Porciju skaits
8.	instructions	TEXT		Pagatavošanas soļi, lai realizētu recepti
9.	user_id	INT	-	Ārējā atslēga jeb identifikators tabulai “users”

Starptabula “recipe_products” ir saistīta ar tabulu “recipes” caur lauku “recipe_id” un ar tabulu “products” caur lauku “product_id”. Tā satur informāciju (sk. 5.8. tabulu) kā identifikatori un daudzums.

5.8.tabula

Tabulas “recipe_products” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	product_id	INT	-	Ieraksta identifikators un ārējā atslēga jeb identifikators tabulai “products”
2.	recipe_id	INT	-	Ieraksta identifikators un ārējā atslēga jeb identifikators tabulai “recipes”
3.	amount	DOUBLE	-	Pārtikas produkta daudzums

Tabula “recipe_type” ir saistīta ar tabulu “recipe_recipe_types”. Tā satur informāciju (sk. 5.9. tabulu) kā identifikators un nosaukums.

5.9.tabula

Tabulas “recipe_types” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	recipe_type_id	INT	-	Ieraksta identifikators
2.	name	VARCHAR	30	Receptes veida nosaukums

Starptabula “recipe_recipe_types” ir saistīta ar tabulu “recipes” caur lauku “recipe_id” un ar tabulu “recipe_types” caur lauku “recipe_type_id”. Tā satur informāciju (sk. 5.8. tabulu) kā identifikatori un pievienošanas datums.

5.10.tabula

Tabulas “recipe_recipe_type” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	recipe_type_id	INT	-	Ieraksta identifikators un ārējā atslēga jeb identifikators tabulai “recipe_types”
2.	recipe_id	INT	-	Ieraksta identifikators un ārējā atslēga jeb identifikators tabulai “recipes”
3.	created_at	DATETIME	-	Receptēs attiecīgo recepšu veidu pievienošanas datums

Tabula “recipe_categories” ir saistīta ar tabulu “recipe_recipe_categories”. Tā satur informāciju (sk. 5.11. tabulu) kā identifikators un nosaukums.

Tabulas “**recipe_categories**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	recipe_category_id	INT	-	Ieraksta identifikators
2.	name	VARCHAR	30	Receptes kategorijas nosaukums

Starptabula “recipe_recipe_categories” ir saistīta ar tabulu “recipes” caur lauku “recipes” un ar tabulu “recipe_categories” caur lauku “recipe_category_id”. Tā satur informāciju (sk. 5.12. tabulu) kā identifikatori un pievienošanas datums.

Tabulas “**recipe_recipe_categories**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	recipe_category_id	INT	-	Ieraksta identifikators un ārējā atslēga jeb identifikators tabulai “recipe_categories”
2.	recipe_id	INT	-	Ieraksta identifikators un ārējā atslēga jeb identifikators tabulai “recipes”
3.	created_at	DATETIME	-	Receptēs attiecīgo recepšu kategoriju pievienošanas datums

Tabula “folders” ir saistīta ar tabulu “folder_recipes” un “users” caur lauku “user_id”. Tā satur informāciju (sk. 5.13. tabulu) kā identifikators un nosaukums.

Tabulas “**folders**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	folder_id	INT	-	Ieraksta identifikators
2.	name	VARCHAR	30	Mapes nosaukums
3.	user_id	INT	-	Ārējā atslēga jeb identifikators tabulai “users”

Starptabula “folder_recipes” ir saistīta ar tabulu “recipes” caur lauku “recipe_id” un ar tabulu “folders” caur lauku “folder_id”. Tā satur informāciju (sk. 5.14. tabulu) kā identifikatori un pievienošanas datums.

Tabulas “**folder_recipes**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	folder_id	INT	-	Ieraksta identifikators un ārējā atslēga jeb identifikators tabulai “folders”
2.	recipe_id	INT	-	Ieraksta identifikators un ārējā atslēga jeb identifikators tabulai “recipes”
3.	created_at	DATETIME	-	Mapēs attiecīgo recepšu pievienošanas datums

Tabula “shopping_lists” ir saistīta ar tabulu “shopping_list_products” un ar tabulu “household” caur lauku “household_id”. Tā satur informāciju (sk. 5.15. tabulu) kā identifikators un nosaukums.

Tabulas “**shopping_lists**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	shopping_list_id	INT	-	Ieraksta identifikators
2.	name	VARCHAR	30	Iepirkšanās saraksta nosaukums
3.	household_id	DOUBLE	-	Ārējā atslēga jeb identifikators tabulai “households”

Starptabula “shopping_list_products” ir saistīta ar tabulu “shopping_lists” caur lauku “shopping_list_id” un ar tabulu “products” caur lauku “product_id”. Tā satur informāciju (sk. 5.14. tabulu) kā identifikatori un pievienošanas datums.

Tabulas “**shopping_list_products**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	shopping_list_id	INT	-	Ieraksta identifikators un ārējā atslēga jeb identifikators tabulai “shopping_list”
2.	product_id	INT	-	Ieraksta identifikators un ārējā atslēga jeb identifikators tabulai “products”
3.	amount	DOUBLE	-	Pārtikas produkta daudzums iepirkšanās sarakstā
4.	created_at	DATETIME	-	Iepirkšanās sarakstā attiecīgo pārtikas produktu pievienošanas datums

Tabula “households” ir saistīta ar tabulu “users”, “shopping_lists” un ar tabulu “household_products”. Tā satur informāciju (sk. 5.16. tabulu) kā identifikators un nosaukums.

5.17.tabula

Tabulas “**households**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	household_id	INT	-	Ieraksta identifikators
2.	name	VARCHAR	30	Mājsaimniecības nosaukums

Starptabula “household_products” ir saistīta ar tabulu “household” caur lauku “household_id” un ar tabulu “products” caur lauku “product_id”. Tā satur informāciju (sk. 5.18. tabulu) kā identifikatori un pievienošanas datums.

5.18.tabula

Tabulas “**household_products**” struktūra

Nr.	Lauka nosaukums	Tips	Izmērs	Apraksts
1.	household_id	INT	-	Ieraksta identifikators un ārējā atslēga jeb identifikators tabulai “households”
2.	product_id	INT	-	Ieraksta identifikators un ārējā atslēga jeb identifikators tabulai “products”
3.	amount	DOUBLE	-	Mājsaimniecības pārtikas produkta daudzums
4.	expiration_date	DATETIME	-	Mājsaimniecības pārtikas produkta derīguma termiņš

6. LIETOTĀJA CEĻVEDIS

6.1. Sistēmas prasības aparatūrai un programmatūrai

Lai veiksmīgi palaistu un darbinātu izstrādāto programmsistēmu, lietotāja datoram jāatbilst noteiktām programmatūras un aparatūras prasībām. Sistēma ir tīmekļa lietojumprogramma, kas izstrādāta ērtai lietošanai visām ierīcēm, ja tā atbilst sekojošajām prasībām:

- **Ieteicamās operētājsistēmas** (nepieciešams, lai operētājsistēma atbalsta Docker un Docker Compose darbību).
 - Windows 10 vai jaunāka versija.
 - Linux (piemēram, Ubuntu 20.04 vai jaunāka).
 - macOS (jaunākās stabilās versijas).
- **Tīmekļa pārlūks** (ar JavaScript un ES6+ standarta atbalstu).
 - Google Chrome versija 100 vai jaunāka.
 - Mozilla Firefox versija 100 vai jaunāka.
 - Microsoft Edge versija 100 vai jaunāka.
 - Safari versija 15 vai jaunāka.
- **Minimālā aparatūra.**
 - Procesors: 2 kodolu CPU.
 - Operatīvā atmiņa (RAM): 4 GB (ieteicams 8 GB, lai nodrošinātu stabilu Docker darbību).
 - Brīvā vieta diskā: 5-10 GB.
- **Programmatūra** (sistēmas palaišanai lokāli vai testa vidē).
 - Docker Desktop (versija 20.10 vai jaunāka).
 - Node.js (ieteicams 18.x vai jaunāks).
 - Npm.
 - Composer.
 - PHP versija jaunāka par 8.3.+.
 - Git (versiju kontrolei).

6.2. Sistēmas instalācija un palaišana

Šajā sadaļā aprakstīta sistēmas palaišana un piekļuves nodrošināšana lietotājiem. Sistēma var tikt darbināta gan lokālajā vidē izstrādes un testēšanas nolūkos, gan tā ir izveidota hostingā un ir pieejama internetā.

6.2.1. Sistēmas piekļūšana lokālā vidē

Lai sāktu sistēmas instalāciju lokālā vidē ir nepieciešamas sekojošās tehnoloģijas uz izvēlētās sistēmas:

- Docker Desktop.
- Git.
- Node.js.
- Npm.
- Composer.
- PHP.
- PhpStorm (Visual studio Code, vai citi koda redaktori).

Tālāk ir detalizēti izklāstītas visas nepieciešamās darbības, lai korekti instalētu un palaistu sistēmu.

1. Projekta klonēšana no GitHub.

1.1. Koda redaktorā atveriet termināli un izpildiet komandu (sk. 6.1.att.).

```
git clone https://github.com/AliseToca/KvDarbs
```

6.1.att. Projekta klonēšanas komanda

1.2. Ieejiet projekta mapē (sk. 6.2.att.).

```
cd KvDarbs
```

6.2.att. Projekta mapes ieiešanas komanda

2. Vides konfigurācijas faila iestatīšana.

2.1. Izveidojot .env failu no .env.example faila informācijas (sk. 6.3.att.).

```
cp .env.example .env
```

6.3.att. .env faila izveides komanda

- 2.2. Atver .env failu un pārliedziniet, ka datubāzes iestatījumi atbilst zemāk attēlotajiem datiem (sk. 6.4.att.).

```
DB_CONNECTION=mysql
DB_HOST=mysql
DB_PORT=3306
DB_DATABASE=laravel
DB_USERNAME=sail
DB_PASSWORD=password
```

6.4.att. .env faila izveides komanda

3. Laravel Sail palaišana.

- 3.1. Composer atkarību instalācija (sk. 6.5.att.).

```
composer install
```

6.5.att. Composer atkarības instalācijas komanda

- 3.2. Atverat DockerDesktop un izpildiet Laravel Sail palaišanas komandu (sk. 6.6.att.).

```
./vendor/bin/sail up -d
```

6.6.att. Laravel Sail palaišanas komanda

4. CubeAgency veidņa instalācija.

- 4.1. CubeAgency instalācijas veidņa palaišana (sk. 6.7.att.).

```
./vendor/bin/sail artisan cube-agency:install
```

6.7.att. CubeAgency instalācijas veidņa palaišanas komanda

- 4.2. Ievadiet datubāzes uzstādīšanas iestatījumus, atzīmējot uz jautājumu “Yes” (sk. 6.8.att.).

```
Are you developing a new application?
Yes
```

6.8.att. Datubāzes uzstādīšanas iestatījums

- 4.3. Ievadiet datubāzes uzstādīšanas iestatījumus, ievadot informāciju no 2.2.punkta minētās informācijas (sk. 6.8.att.).

```
Initializing database setup...

Enter a database name _____
laravel

What is your MySQL host? _____
mysql

What is your MySQL port? _____
3306

What is your MySQL username? _____
root

What is your MySQL password? _____
.....

Create new database "laravel"?
● Yes / ○ No
```

6.8.att. Datubāzes uzstādīšanas iestatījums atbilstši 2.2.punkta informācijai

- 4.4. Apstipriniet zemāk redzamajā logā ievadi (sk. 6.9.att.).

```
Generating permissions and policies for all entities...

Which panel do you want to generate permissions/policies for?
admin
```

6.9.att. Ievades loga ievades apstiprināšana

- 4.5. Izveidojiet administratora kontu aizpildot sekojošo informāciju, parole - admin123 (sk. 6.10.att.).

```
Creating super admin...

Name _____
SiteAdmin

Username _____
siteAdmin

Email address _____
majasreceptes@example.com

Password _____
.....
```

6.10.att. Admin konta izveides informācija

- 4.6. Nesaistat repozitoriju ar organizāciju un atstājiat tukšu logu. (sk. 6.11.att.).

```
Enter your new repository (cube/new-project-name) _____
```

6.11.att. Repozitorija sasaistīšanas ievades loga ievaddati

5. Datubāzes migrācija.

- 5.1. Izpildiet datubāzes migrācijas komandu (sk. 6.12.att.).

```
./vendor/bin/sail artisan migrate --seed
```

6.12.att. Datubāzes migrācijas un sākotnējo datu ievades komanda

6. Node.js atkarības instalācija un palaišana

- 6.1. Ienistalējiet Npm pakotni ar sekojošo komandu (sk. 6.13.att.).

```
./vendor/bin/sail npm install
```

6.13.att. Npm pakotnes instalēšanas komanda

- 6.2. Palaidiet ieinstalēto pakotni (sk. 6.14.att.).

```
./vendor/bin/sail npm run dev
```

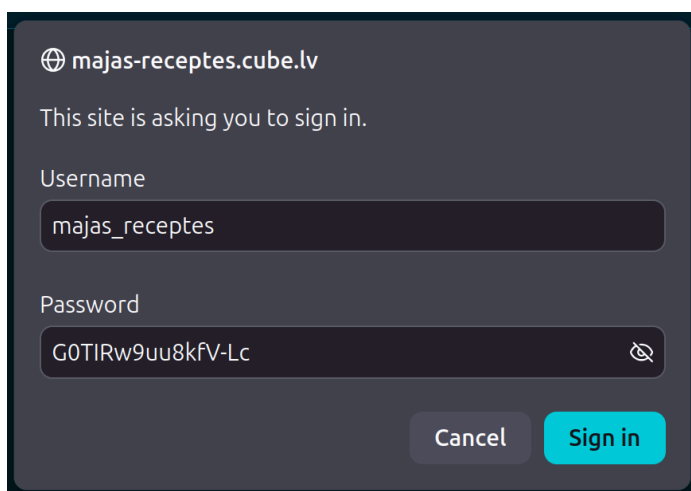
6.14.att. Npm palaišanas komanda

Veiksmīgi beidzot izpildīt visus iepriekš minētos soļus, sistēma tiks veiksmīgi instalēta un palaista. Izvēlētajā tīmekļa vietnē, apmeklējiet <http://localhost:8000>, lai aplūkotu sistēmu.

6.2.2. Sistēmas piekļūšana hostingā

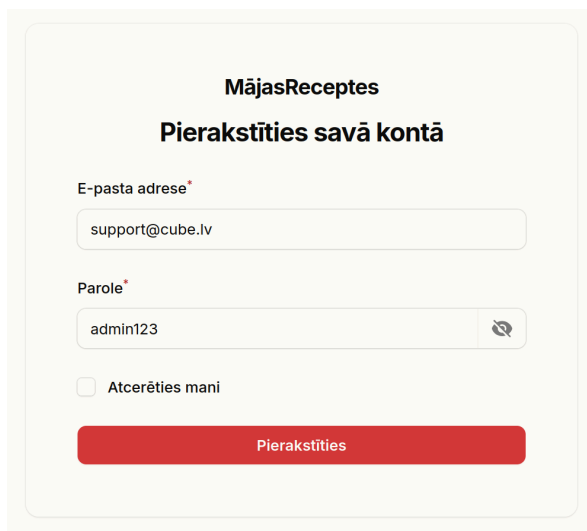
Lai sistēmai, kas izvietota konkrētā tīmekļa domēna adresē, piekļūtu un tiktu pie sistēmā svarīgajām lomām ir jāizpilda sekojošie soļi.

1. Apmeklējiet saiti **majas-receptes.cube.lv/lv**.
2. Pieprasījuma logā ievadiet **lietotājevārdu: majas_receptes** un **paroli: G0TIRw9uu8kfV-Lc** (sk. 6.15.att.).



6.15.att. Pieprasījuma loga ievaddati

3. Tālāk apmeklējiet saiti **<https://majas-receptes.cube.lv/admin/login>**.
4. Pieslēgšanās formā ievadiet e-pastu: **support@cube.lv** un paroli: **admin123** (sk. 6.16.att.).

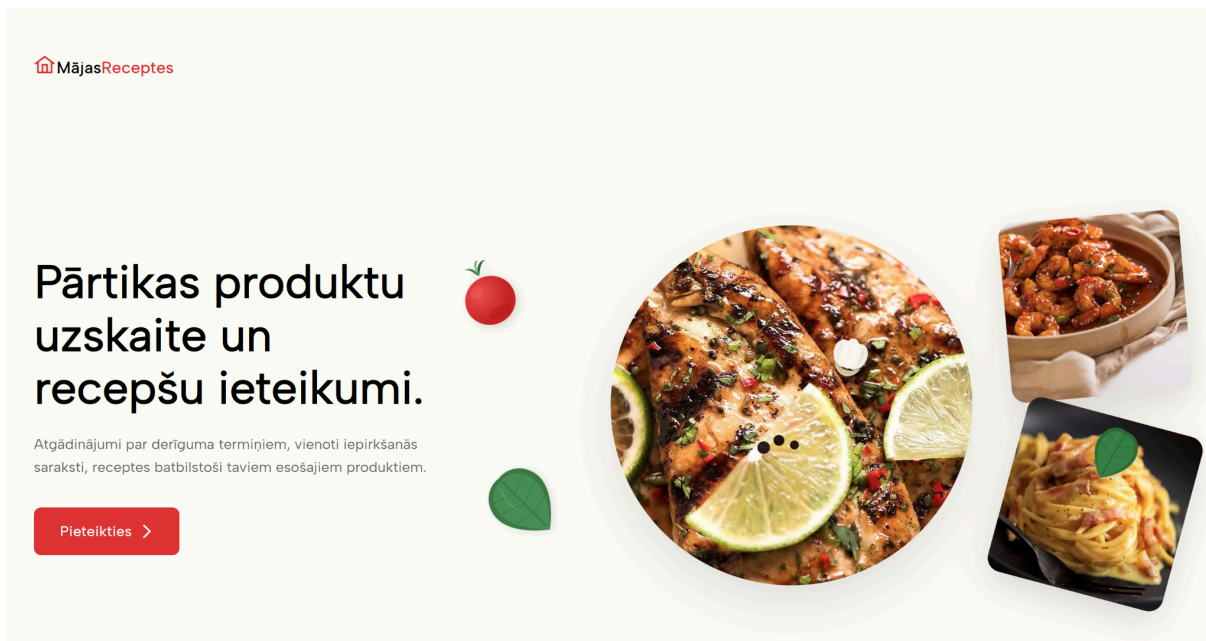


6.16.att. Administratora konta formas ievaddati

6.3. Programmas apraksts

Galvenā lapa.

Lietotājam, apmeklējot mājaslapu būs redzams galvenās lapas logs (sk.6.17.att), kur būs redzama informācija par mājaslapu un poga “Pieslēgties”. Uzklīkšķinot uz tās lietotājs tiks novirzīts uz pieslēgšanās logu un tā formu.



6.17.att. Galvenais lapas ekrānsskats

Pieslēgšanās forma (Autentifikācija).

Lietotājam, kurš jau ir reģistrēts sistēmā, ir iespēja autentificēties sistēmā, izmantojot pieslēgšanās formu (sk. 6.18.att.). Šis ekrāns paredzēts lietotāja autentifikācijai, ievadot derīgu e-pastu (vai lietotājvārdu) un paroli.

The image shows a login form titled 'Pieteikšanās'. It has two input fields: 'E-pasts' and 'Parole'. The 'Parole' field has an eye icon to toggle visibility. Below the fields is a red button labeled 'Pieteikties'. Underneath the button is a link that says 'Aizmirsāt paroli?'. At the bottom of the form, there is a link that says 'Nav konta? Reģistrācija'.

6.18.att. Pieslēgšanās forma

Reģistrācijas forma (Autentifikācija).

Lietotājam, kurš nav reģistrēts sistēmā, ir iespējama izveidot lietotāja konta, izmantojot reģistrācijas formu (sk. 6.19.att.). Šis ekrāns paredzēts lietotāja konta izveidei, ievadot vārdu, e-pastu, lietotājvārdu, paroli un tās apstiprinājumu.

The image shows a registration form titled "Reģistrācija". It contains five input fields: "Vārds" (Name), "E-pasts" (Email), "Lietotājvārds" (Username), "Parole" (Password), and "Paroles apstiprinājums" (Password Confirmation). Each field has a light gray placeholder box. The "Parole" and "Paroles apstiprinājums" fields have an eye icon on the right side to toggle visibility. Below the fields is a red button labeled "Reģistrēties". At the bottom, there is a link: "Jau ir konts? [Pieteikšanās](#)".

6.19.att. Reģistrācijas forma

Navigācijas (galvene)

Lapas augšdaļā ir galvenes navigācijas josla (sk. 6.20.att.), kas ļauj lietotājam ātri piekļūt svarīgākajām sadaļām - receptes, mājsaimniecība, iepirkšanās saraksts un profils ar uzniestošu izvēlni zem tā.

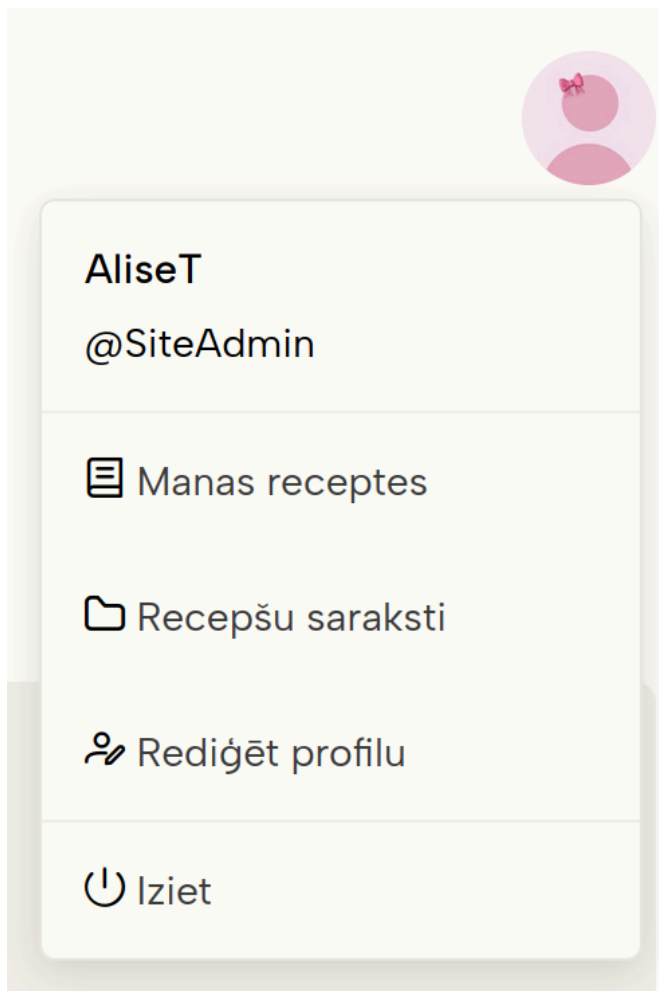


6.20.att. Galvenes navigācijas joslas ekrāna skats

Navigācija (profils).

Noklikšķinot uz lietotāja profila ikonas galvenē, lietotājam parādās papildus izvēlnes logs (sk. 6.21.att.), kas satur informāciju par lietotāja vārdu un lietotājvārdu, navigāciju uz lapas citām daļām - manas receptes, recepšu saraksti, rediģēt profilu - un opciju izrakstīties no

lietotāja konta. Iespēja izvēlēties apmeklēt lapas daļas vai izrakstīties no konta var klikšķinot uz attiecīgo uzrakstu. Logu var aizvērt, noklikšķinot ārpus tā vai uz profila ikonas.



6.21.att. Profila izvēlnes logs

Navigācija (kājene).

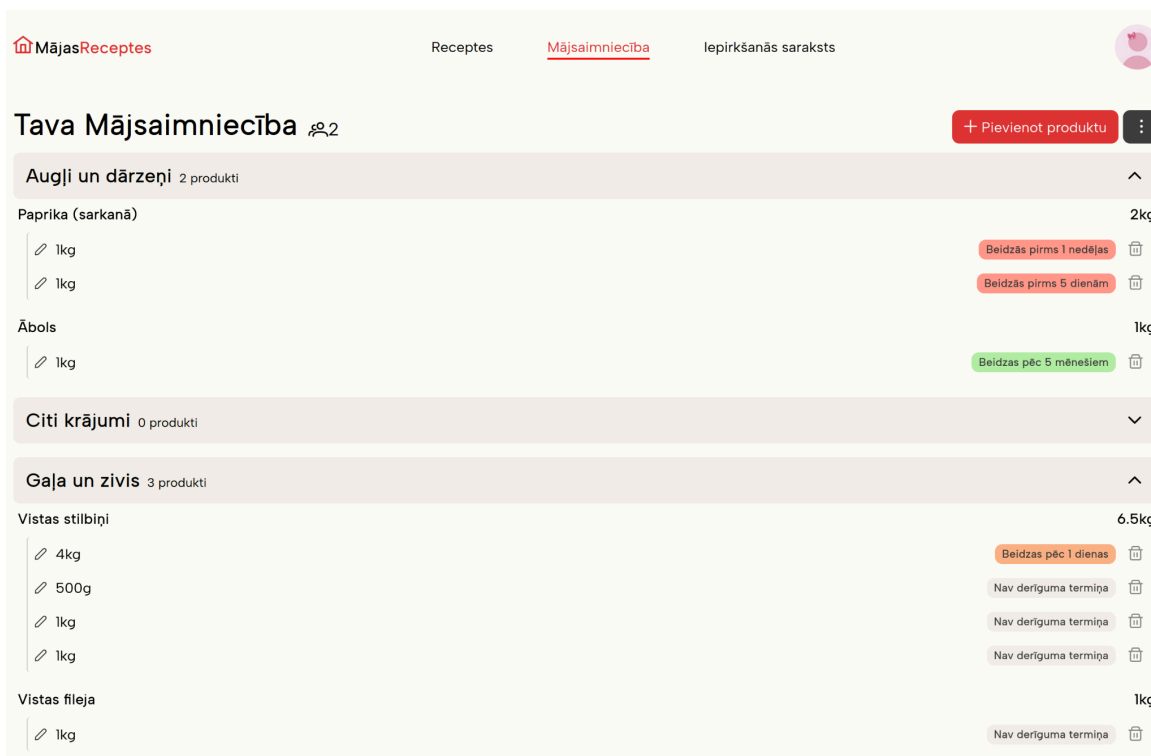
Lapas apakšdaļā ir kājene ar galvenas un profila navigācijas joslām (sk. 6.22.att.), kā arī lapas nosaukums ar sociālo tīklu novirzēm. Klikšķinot uz navigācijas nosaukumu, lietotājs tiks novirzīts uz attiecīgo lapu.



6.22.att. Kājenes navigācijas josla

Mājsaimniecības skats.

Mājsaimniecības lapas daļā (sk. 6.23.att.) lietotājs redz savu mājsaimniecību - nosaukumu, cilvēku skaitu tajā, kategorizētos produktu sarakstus, iespēja pievienot produktus un atvērt papildus loga izvēlnes darbības.

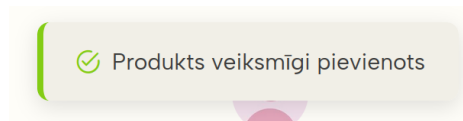


6.23.att. Lietotāja mājsaimniecības skats

Noklikšķinot uz iepriekšējā skatā redzamo “+ Pievienot produktu” pogu, lietotājam atveras produkta pievienošanas logs (sk. 6.24.att.). Logu ir iespējams aizvērt nospiežot ‘X’ vai klikšķinot ārpus tā. Lietotājs nevar ievadīt informāciju par produktu mērvienību līdz netiek izvēlēts produkts.

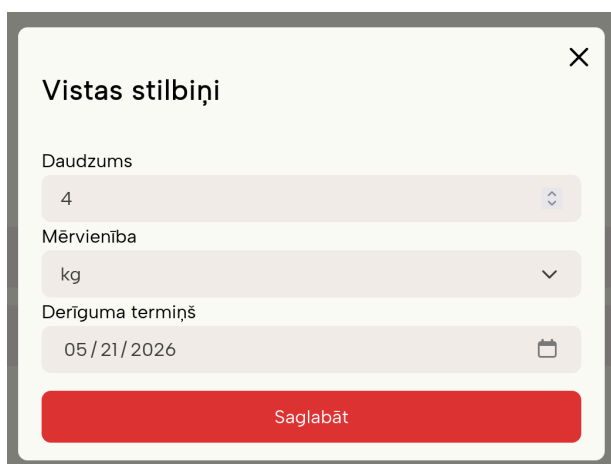
6.24.att. Produkta pievienošanas logs

Ievadot visu nepieciešamo informāciju un nospiežot “Pievienot” lietotājs ir veiksmīgi pievienojis produktu un tiek parādīts paziņojums (sk. 6.25.att.).



6.25.att. Paziņojums par veiksmīga produkta pievienošanu.

Noklikšķinot uz kategorizēto produktu mājsaimniecībā atveras produkta informācijas rediģēšanas logs (sk. 6.26.att.). Lietotājam ir iespēja rediģēt produkta daudzumu, mērvienību un derīguma termiņu. Nospiežot pogu “Saglabāt” tiks atjaunināta produkta informācija.



6.26.att. Produkta rediģēšanas formas logs

Noklikšķinot uz kategorizēto produkta miskastes ikonas tiks atvērts dzēšanas apstiprinājuma logs (sk 6.27.att.). Lietotājam noklikšķinot uz “Dzēst” produkts tiks dzēsts no mājsaimniecības, bet spiežot “Atcelt” darbība netiks veikta un logs aizvērsies.



6.27.att. Produkta dzēšanas apstiprinājuma logs

Mājsaimniecības papildu darbības logs (mājsaimniecības saimnieks)

Noklikšķinot uz trīs vertikālo punktu pogu tiek atvērta izvēlne (sk. 6.28.att.) ar papildus mājsaimniecības darbībām - uzaicini lietotāju mājsaimniecībai, iestatījumi, pamest

mājsaimniecību. Iestatījumos ir iespēja mainīt mājsaimniecības nosaukumu un pašreizējo mājsaimniecības dalībnieku lomas.



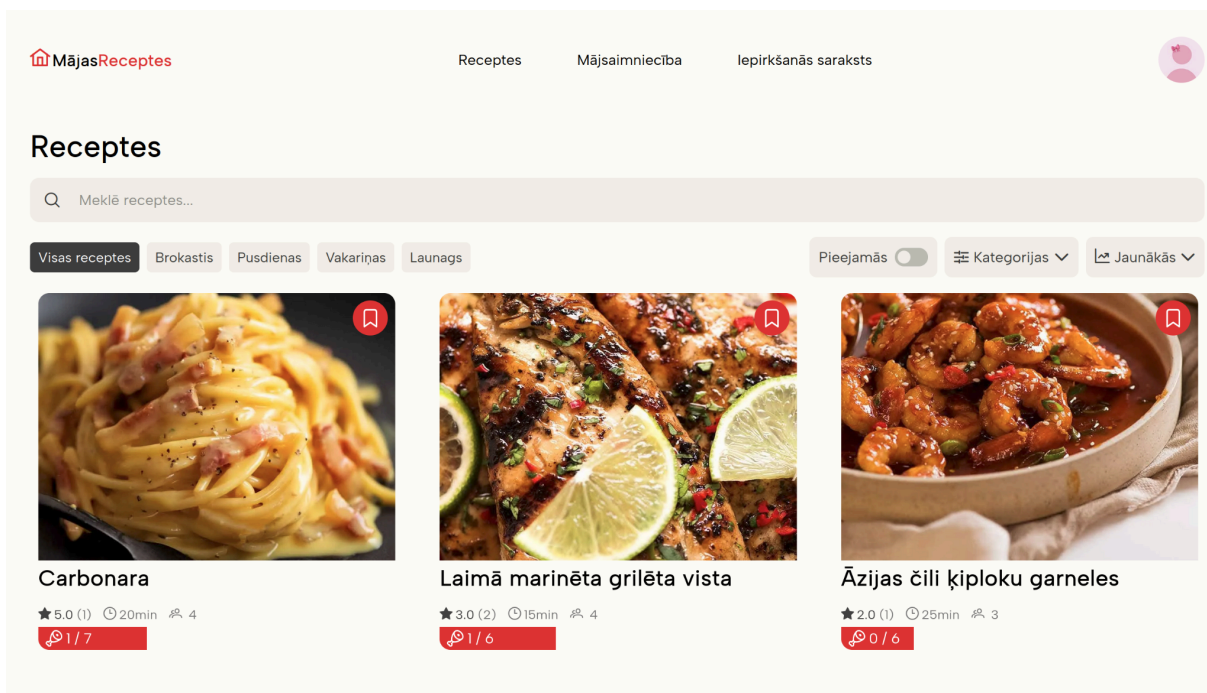
6.28.att. Mājsaimniecības papildus darbību izvēlne

Mājsaimniecības papilu darbības logs (mājsaimniecības dalībnieks)

Noklikšķinot uz trīs vertikālo punktu pogu tiek atvērta izvēlne tikai ar vienu iespēju - pamest mājsaimniecību.

Receptes galvenais skats

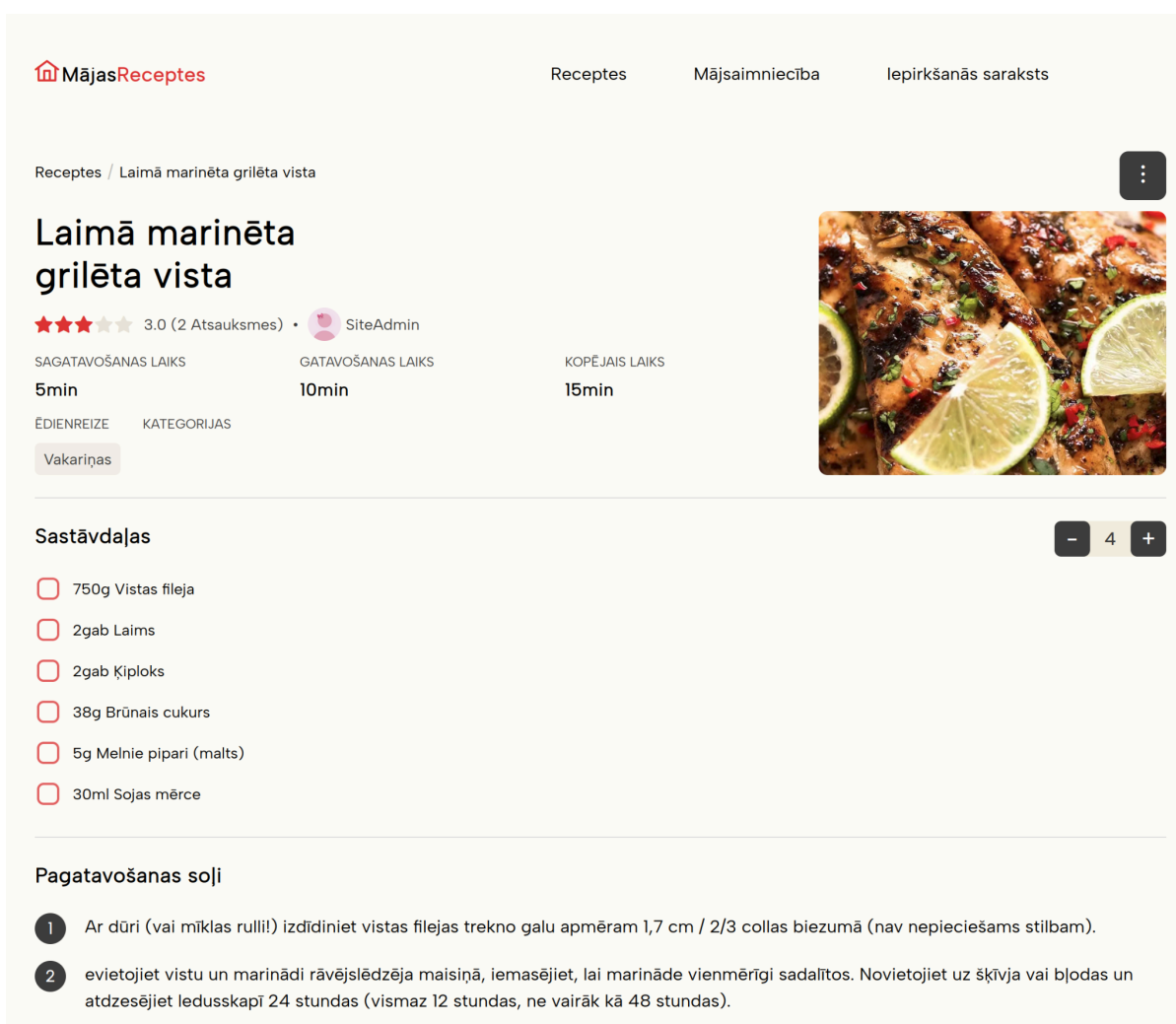
Lapas sadaļā receptes (sk. 6.29.att.) ir redzamas visas sistēmas receptes kā arī iespēja tās meklēt, filtrēt pēc ēdienreizes, kategorijas un pieejamības (mājsaimniecība ir pietiekami produkti receptei) un kārtošana pēc izveides, datuma, augstākā novērtējuma un visvairāk novērtēto.



6.29.att. Receptes galvenais ekrāna skats

Receptes individuālais skats

Noklikšķinot uz receptes ir pieejams receptes skats (sk. 6.30.att.). Receptes skatā ir informācija par receptes pamatinformāciju - nosaukumu, novērtējumu, autoru, ēdienreizes, kategoriju, pagatavošanas, sagatavošanas un kopējiem laikiem, nepieciešajām sastāvdaļām, pagatavošanas soļiem un atsauksmēm. Augšā ir navigācijas josla, kas norāda pašreizējo recepti un iespēju iet atpakaļ uz kopējo recepšu lapu. Ir iespēja mainīt porciju skaitu un atzīmēt jau izmantotās sastāvdaļas.

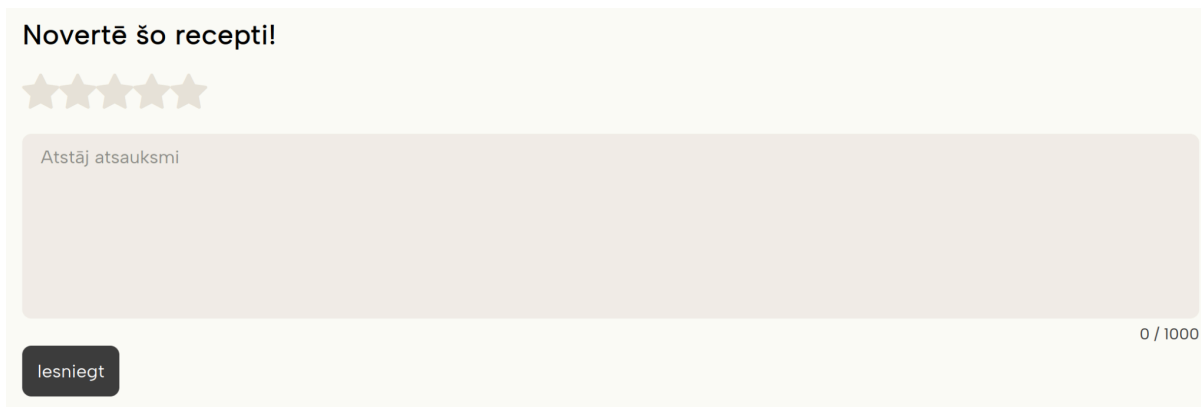


The screenshot displays the 'Laimā marinēta grilēta vista' recipe page. At the top, there's a navigation bar with 'MājasReceptes' and links to 'Receptes', 'Mājsaimniecība', and 'Iepirkšanās saraksts'. The breadcrumb trail shows 'Receptes / Laimā marinēta grilēta vista'. The recipe title is 'Laimā marinēta grilēta vista' with a 3.0 star rating from 2 reviews by 'SiteAdmin'. It lists three time periods: 'SAGATAVOŠANAS LAIKS 5min', 'GATAVOŠANAS LAIKS 10min', and 'KOPĒJAIS LAIKS 15min'. The 'ĒDIENREIZE' is 'Vakariņas'. A photo of the dish is shown on the right. Below the title, the 'Sastāvdaļas' (Ingredients) section lists: 750g Vistas fileja, 2gab Laims, 2gab Ķiploks, 38g Brūnais cukurs, 5g Melnie pipari (malts), and 30ml Sojas mērce. The 'Pagatavošanas soļi' (Preparation steps) section shows two steps: 1. 'Ar dūri (vai mīklas rulli!) izdīdiniet vistas filejas trekno galu apmēram 1,7 cm / 2/3 collas biezumā (nav nepieciešams stilbam).', and 2. 'evietojiet vistu un marinādi rāvēslēdzēja maisiņā, iemasējiet, lai marināde vienmērīgi sadalītos. Novietojiet uz šķīvja vai bļodas un atdzesējiet ledusskapī 24 stundas (vismaz 12 stundas, ne vairāk kā 48 stundas).'

6.30.att. Receptes galvenais ekrāna skats

Lietotājam zem receptes ir iespēja aizpildīt atsauksmes formu (sk. 6.31.att.). Ir piecas zvaigznes, kur lietotājam, tās atzīmējot, ir iespēja novērtēt recepti skalā no 1-5. Zem tā ir teksta logs, kur lietotājs var iesniegt atsauksmi 1000 rakstzīmju ierobežojumā. Nospiežot pogu 'Iesniegt' lietotāja atsauksme tiek izveidota un tā tiek parādīta zem receptes ar citu

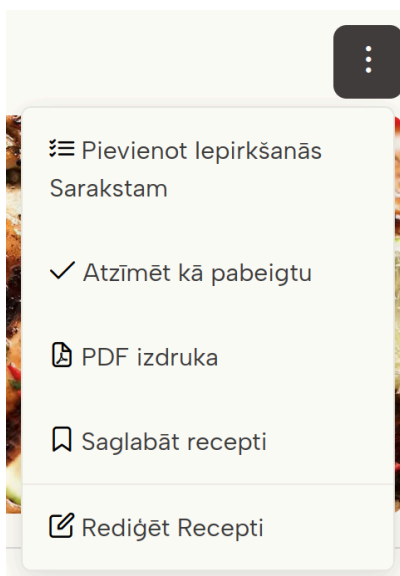
lietotāju atsauksmēm. Kā arī lietotāja novērtējums tiek pieskaitīts receptes kopējam novērtējumam un aprēķināts receptes vidējais aritmētiskais novērtējums.



6.31.att. Atsauksmes izveides forma

Lietotājam ir iespēja dzēst **savas** izveidotās atsauksmes zem receptes. Nospiežot miskastes ikonu tiek izvadīts tāds pats apstiprinājuma logs kā mājsaimniecības produkta dzēšanā, kur lietotājam ir iespēja apstiprināt vai atcelt darbību.

Nospiežot uz receptes individuālā skata pogu ar trīs vertikāliem punktiem tiek atvērts receptes izvēlne (sk. 6.32.att.) ar darbībām: pievienot iepirkšanās sarakstam (pēc apstiprināšanas pievienos trūkstošos produktus no receptes), atzīmēt kā pabeigtu (pēc apstiprināšanas noņems no mājsaimniecības receptē izmantoto produktu daudzumu), pdf izdruka, saglabāt recepti un rediģēt recepti (ja lietotājs ir receptes veidotājs).

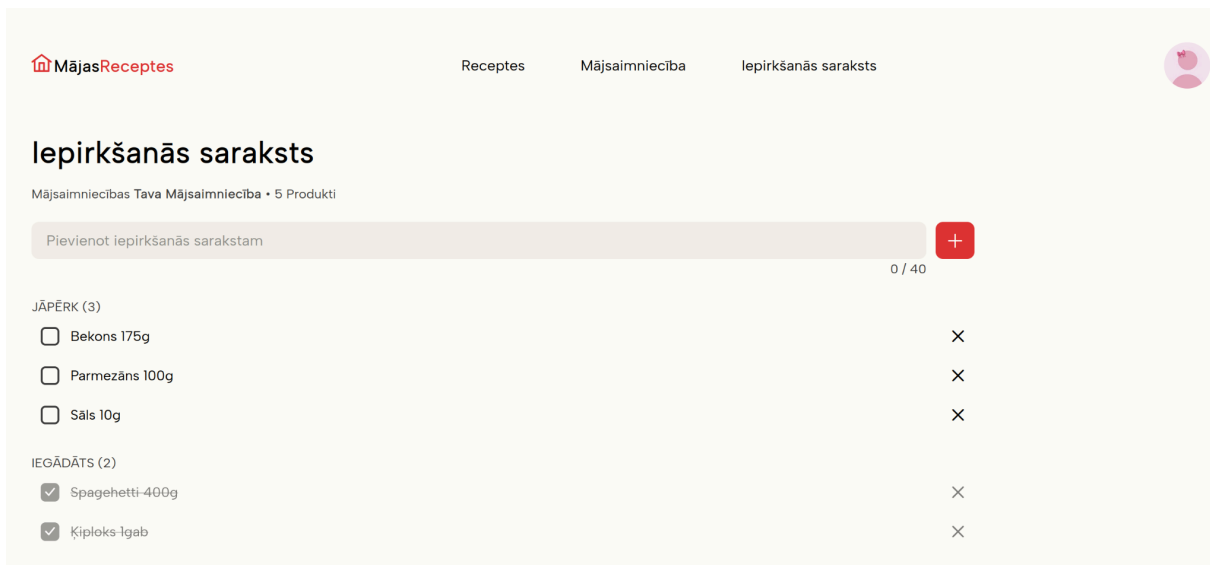


6.32.att. Receptes papildus darbību izvēlnes skats

Iepirkšanās saraksta skats

Lapas sadaļā iepirkšanās saraksts (sk. 6.33.att.) ir redzama informācija par kopējo produktu skaitu, mājsaimniecības nosaukumu, nopērkamo produktu sarakstu un iegādāto produktu sarakstu. Lietotājam ir iespēja pievienot jaunus produktus ievadot to informāciju

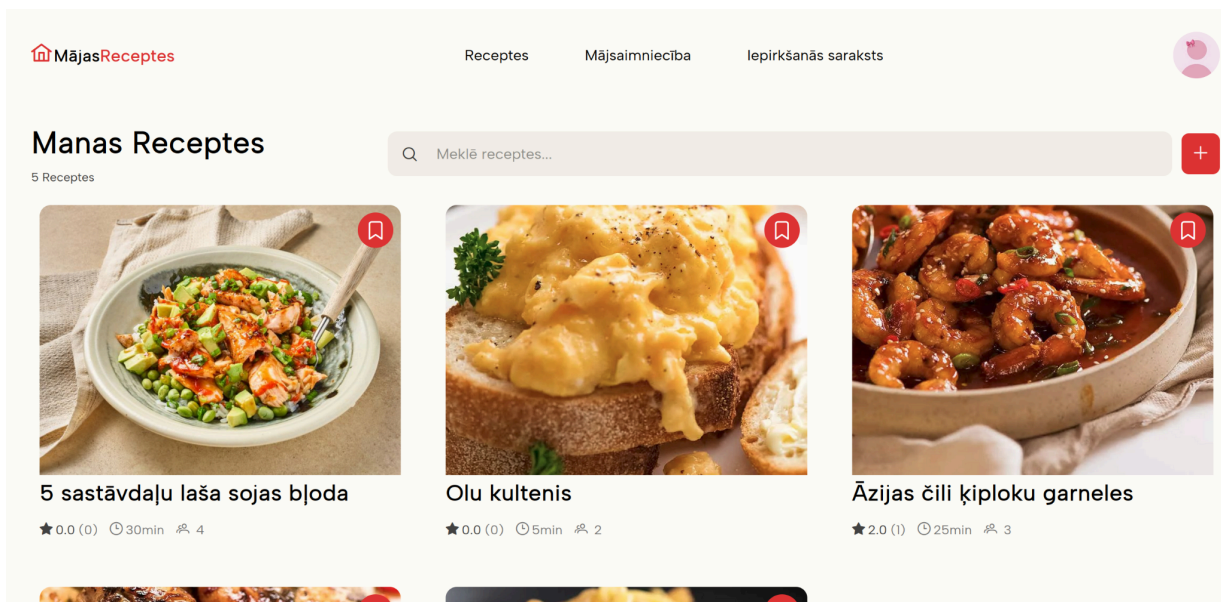
ievades logā un nospiežot pogu “+”, atzīmēt produktus kā iegādātus/neiegādātus, kā arī dzēst tos pilnībā no saraksta nospiežot uz attiecīgā produkta “X”.



6.33.att. Iepirkšanās saraksta galvenais ekrāna skats

Manas receptes skats

Lapas sadaļā manas receptes (sk. 6.34.att.) ir redzamas visas lietotāja izveidotās receptes. Sadaļā ir redzams kopējais recepšu skaits, meklēšanas logs, “+” poga (jaunas receptes izveide) un saraksts ar izveidotajām receptēm. Lietotājam ir iespēja meklēt receptes un arī tās saglabāt.



6.34.att. Manas receptes ekrāna skats

Noklikšķinot uz “+” lietotājs tiek novirzīts uz jaunas receptes izveides lapu (sk. 6.35.att.), kur lietotājs norāda receptes nosaukumu, attēlu, redzamību, ēdienreizi, kategorijas, laikus, sastāvdaļas un pagatavošanas soļus. Lietotājam ievades logos ir iespēja ievadīt

attiecīgo informāciju. Sadaļās “Satāvdaļas” un “Pagatavošanas soļi” noklikšķinot uz “+” tiek pievienots jauna ievades rinda. Poga “Noņemt” to noņems. Lietotājam nospiežot “Saglabāt” recepte tiks izveidota un pievienota kopējā recepšu sarakstā un lietotāja.

Manas Receptes / Izveidot recepti

Izveidot recepti

Nosaukums

0 / 50

Attēls



Browse... No file selected.

Redzamība

Mājsaimniecība

Detaļas

Sagatavošanas laiks

0 h 0 min

Gatavošanas laiks

0 h 0 min

Porcijas

1

Ēdienreize

Brokastis

Pusdienas

Vakariņas

Launags

Kategorijas

Veģetārs

Vegānisks

Āzijas virtuve

Itāļu virtuve

Taizemes virtuve

Makaroni

Rietumu virtuve

Sastāvdaļas

Nosaukums

Meklē produktus...

Daudzums

Mērvienība

Sākumā izvēl

Noņemt

Pagatavošanas soļi

1. Solis

Noņemt

Saglabāt

6.35.att. Daļa no receptes izveides ekrāna skata

No manas receptes skata (sk. 6.34.att.) lietotājs noklikšķinot uz kādo no receptēm sarakstā tiek novirzīts uz receptes rediģēšanas skatu (sk. 6.36.att.). Tam ir tāda paša struktūra kā receptes izveides skatam, tikai ievades lauku vietā ir receptes informācija.

Manas Receptes / Rediģēt "Krēmveida Citron...

Rediģēt Recepti

Dzēst

Nosaukums

Krēmveida Citronu Vistas Krūtiņas

33 / 50

Attēls



Browse... No file selected.

Redzamība

Publiski

Detāļas

Sagatavošanas laiks

0 h 5 min

Gatavošanas laiks

0 h 10 min

Porcijas

4

Ēdienreize

Brokastis

Pusdienas

Vakariņas

Launags

Kategorijas

Veģetārs

Veģānisks

Āzijas virtuve

Itāļu virtuve

Taizemes virtuve

Makaroni

Rietumu virtuve

Sastāvdaļas

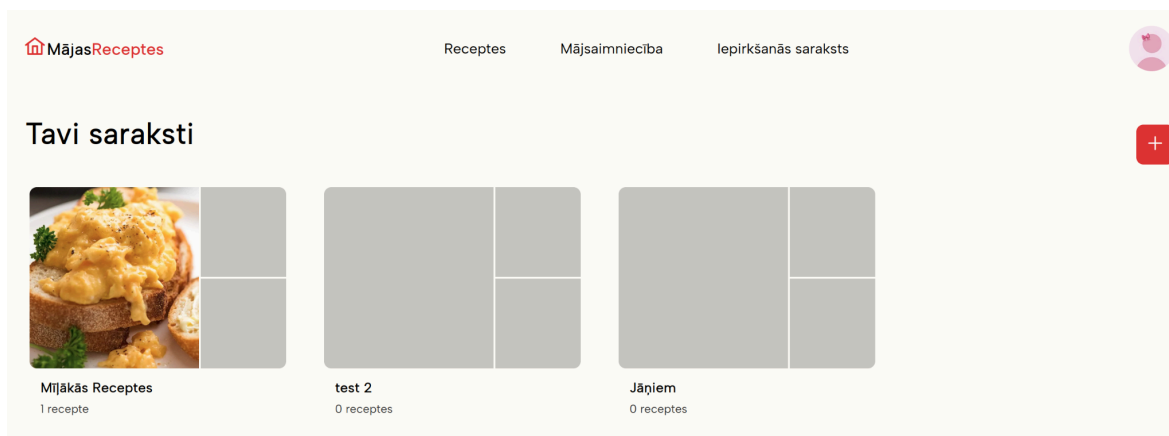
+

Nosaukums	Daudzums	Mērvienība	
Vistas krūtiņas	600	g	Noņemt
Kviešu milti	35	g	Noņemt
Sviests	40	g	Noņemt
Ķiploka daiviņas	2	gab	Noņemt
Vistas buljons	310	ml	Noņemt

6.36.att. Daļa no receptes rediģēšanas ekrāna skata

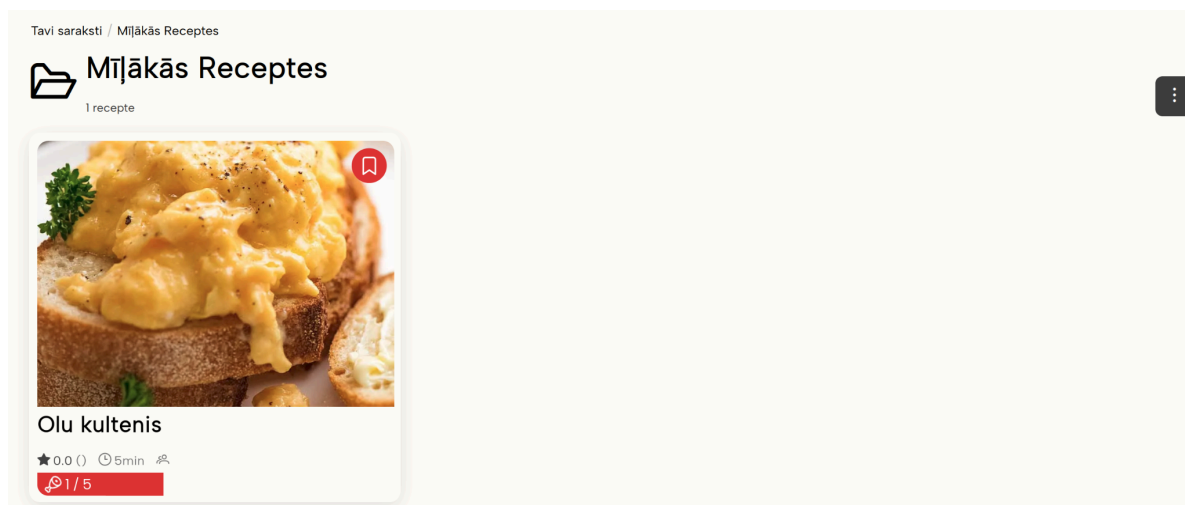
Recepšu sarakstu skats

Lapas sadaļā recepšu saraksti (sk. 6.37.att.) ir redzami visi lietotāja izveidotie recepšu saraksti un to recepšu skaits tajos. Kā arī poga “+”, kas ļauj lietotājam pievienot vēl sarakstus.



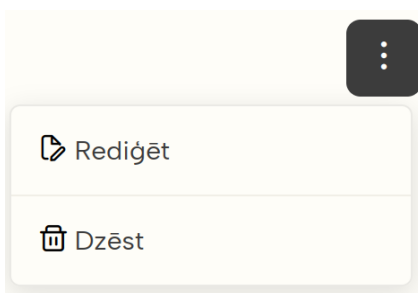
6.37.att. Recepšu sarakstu ekrāna skats

Noklikšķinot uz receptes sarakstu tiek atvērts saraksta individuālais skats (sk. 6.38.att.). Ir redzams saraksta nosaukums, receptes skaits, un recepšu saraksts. Noklikšķinot uz receptes sarakstu tas aizvedīs lietotāju uz receptes individuālo skata lapu (iepriekš aprakstīta).



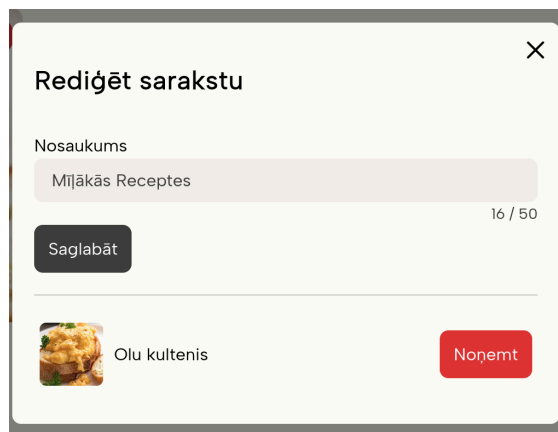
6.38.att. Recepšu saraksta individuālais ekrāna skats

Noklikšķinot uz pogas ar vertikālajiem trīs punktiem atvērs saraksta papildus darbību izvēlni (sk. 6.39.att.).



6.39.att. Receptu saraksta papildus darbību izvēlne

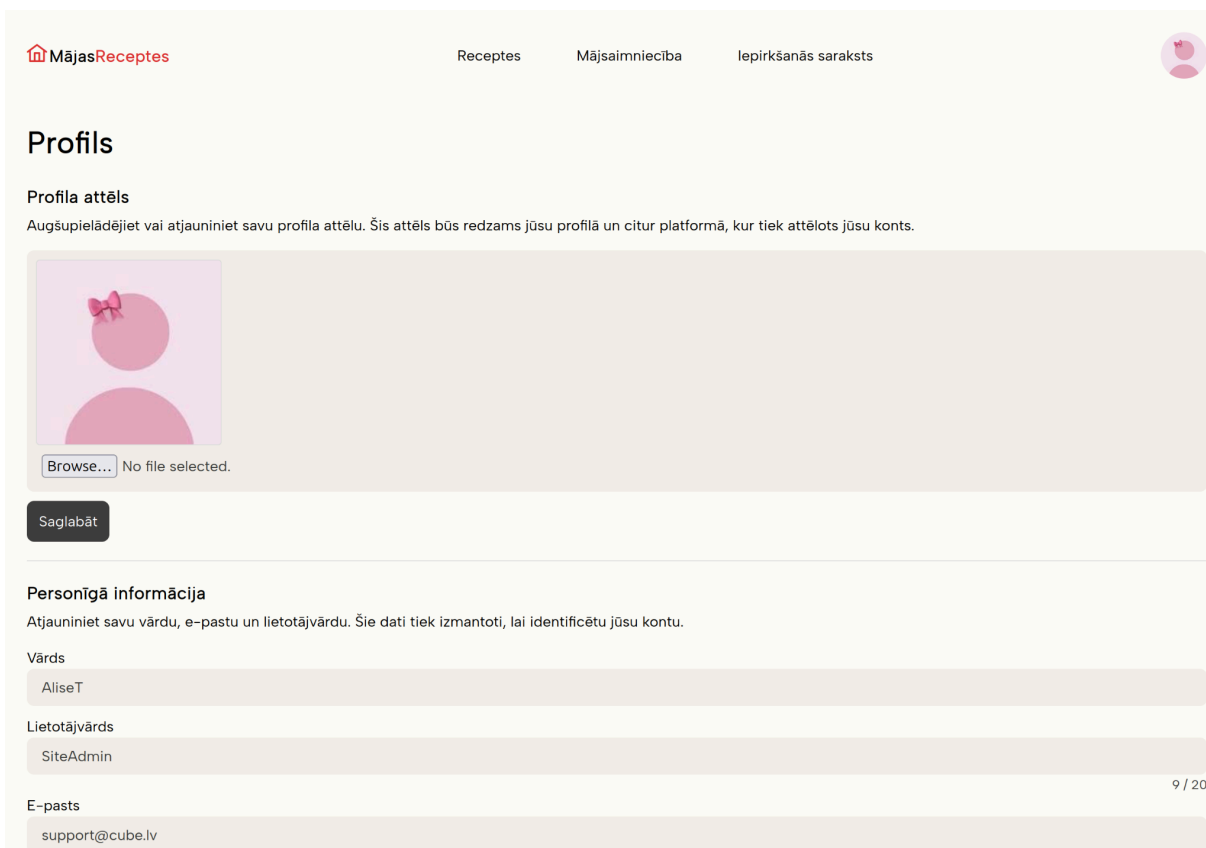
Tajā ir darbības: rediģēt sarakstu un dzēst sarakstu. Izvēloties saraksta dzēšanu parādas apstiprinājuma logs ar iespēju apstiprināt vai atcelt dzēšanas darbību, bet izvēloties rediģēt sarakstu tiks parādīts logs (sk. 6.40.att.) ar iespēju mainīt receptes saraksta nosaukumu un noņemt receptes no saraksta.



6.40.att. Saraksta rediģēšanas logs

Rediģēt profila skats

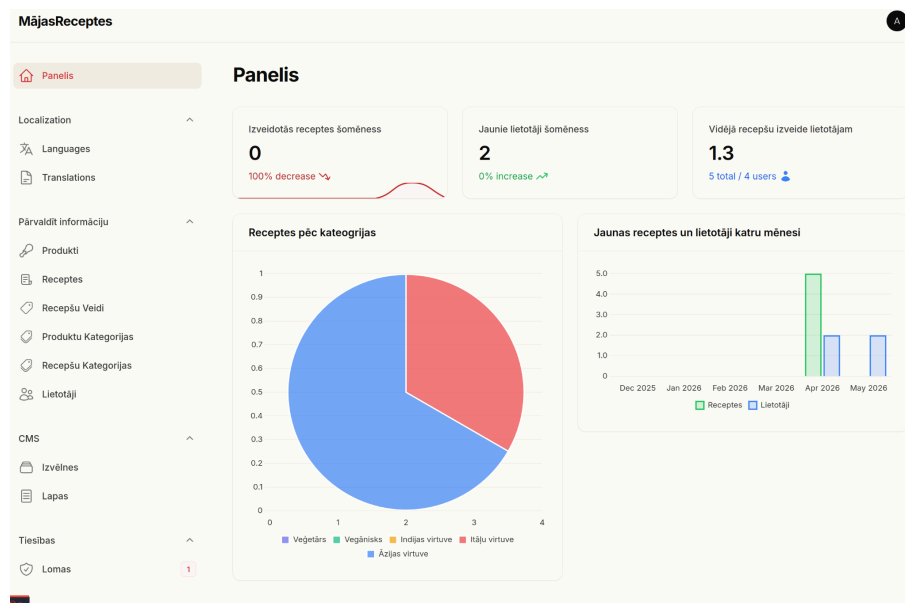
Lapas sadaļā rediģēt profilu (sk. 6.41.att.) ir redzama visa lietotāja profila informācija - vārds, lietotājavārds, e-pasts, profila attēls, parole - , kā arī iespēja dzēst kontu. Lietotājam ir iespēja rediģēt visus eksistējošos ievaddatus aizpildot formas un spiežot pogas “Saglabāt”.



6.41.att. Profila skata ekrāna daļa

Administrācijas panelis

Lietotājiem ar administrācijas tiesībām ir pieejami papildus administrācijas paneļa skati. Ieejot panelī lietotājs ir novirzīts uz galveno lapu (sk. 6.42.att.), kur ir redzama lapas statistika par jaunajiem lietotājiem un receptēm, vidējo recepšu izveidi uz lietotāju, procentuālo receptes daudzumu pēc kategorijām un salīdzinājumu ar jaunajiem lietotājiem un receptēm. Šāna ir redzama navigācijas josla uz visu sistēmai informāciju. Galvenē ir lapas nosaukums, meklēšanas logs un profila ikona.



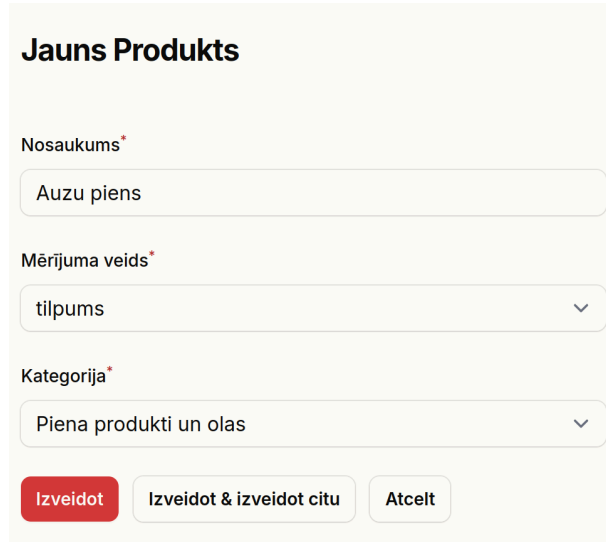
6.42.att. Administratora paneļa galvenais skats

Administratoram ir iespēja apskatīt (sk. 6.43.att.) , rediģēt un dzēst informāciju par sistēmas: produktiem, receptēm, recepšu veidiem, receptes kategorijām, produktu kategorijām un lietotājiem.

Receptes > Saraksts							
Receptes Jauns Recepte							
Meklēt							
☐	Nosaukums	Autors	Ēdienreize	Redzamība	Porcijas	Atsauksmes	Izveidots
☐	5 sastāvdaļu laša sojas bļoda	SiteAdmin	Pusdienas	private	4	0	pirms 2 nedēļ
☐	Olu kultenis	SiteAdmin	Brokastis	household	2	0	pirms 2 nedēļ
☐	Āzijas čili ķiploku garneles	SiteAdmin	Vakariņas	public	3	1	pirms 2 nedēļ
☐	Laimā marinēta grilēta vista	SiteAdmin	Vakariņas	public	4	2	pirms 2 nedēļ
☐	Carbonara	SiteAdmin	Pusdienas	household	4	1	pirms 2 nedēļ
Rāda 1 līdz 5 no 5 rezultātiem					vienā lappusē 10		

6.43.att. Administratora paneļa datu apskatīšanas skats

Administratoram ir iespēja apskatīt, rediģēt (sk. 6.44.att.) un dzēst informāciju par sistēmas: produktiem, receptēm, recepšu veidiem, receptes kategorijām, produktu kategorijām un lietotājiem.

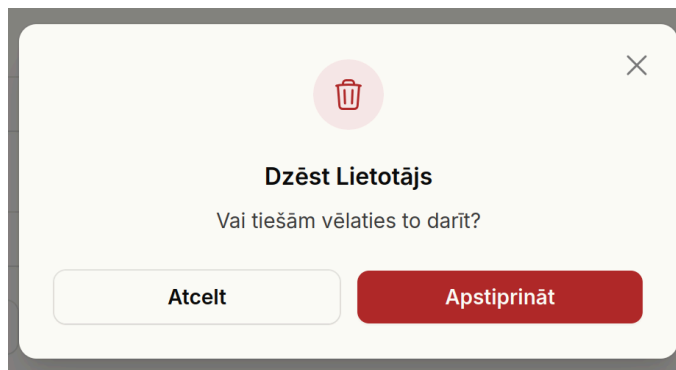


The screenshot shows a form titled "Jauns Produkts" (New Product) with the following fields and buttons:

- Nosaukums*** (Name): A text input field containing "Auzu piens" (Oat milk).
- Mērījuma veids*** (Measurement type): A dropdown menu with "tilpums" (volume) selected.
- Kategorija*** (Category): A dropdown menu with "Piena produkti un olas" (Dairy products and eggs) selected.
- Buttons:** "Izveidot" (Create) in red, "Izveidot & izveidot citu" (Create & create another) in light gray, and "Atcelt" (Cancel) in light gray.

6.44.att. Administratora paneļa ieraksta izveides skats

Administratoram ir iespēja apskatīt, rediģēt un dzēst (sk. 6.45.att.) informāciju par sistēmas: produktiem, receptēm, recepšu veidiem, receptes kategorijām, produktu kategorijām un lietotājiem.



6.45.att. Administratora paneļa ieraksta dzēšanas skats

7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA

Zemāk (sk. 7.1.tabulu) tiek pārbaudīta lietotāju autentifikācijas funkcionalitāte, izvērtējot sistēmas darbību dažādos pieteikšanās scenārijos ar pareiziem un nepareiziem piekļuves datiem.

7.1.tabula

Lietotāja sistēmas pieteikšanās testēšanas piemērs

Lietotāju loma:	viesis		
Prasību identifikators:	PS 2.2.1.		
Prasības apraksts			
Jāļauj lietotājam autentificēties sistēmā un piekļūt sistēmas funkcionalitātēm atbilstoši savām tiesībām, izmantojot formu, kur tiek norādīts lietotāja e-pasts vai lietotājvārds un parole.			
Ievaddati/situācijas apraksts		Sagaidāmais rezultāts	Statuss
1. Lietotājs ievada eksistējošu sistēmas e-pastu vai lietotājvārdu un atbilstošu paroli.		ļauj lietotājam autentificēties sistēmā	pareizi
2. Lietotājs ievada eksistējošu sistēmas e-pastu vai lietotājvārdu, bet neatbilstošu paroli.		neļauj lietotājam autentificēties sistēmā un tiek izvadīts kļūdas paziņojums	pareizi
3. Lietotājs ievada neeksistējošu sistēmas e-pastu vai lietotājvārdu un paroli.		neļauj lietotājam autentificēties sistēmā un tiek izvadīts kļūdas paziņojums	pareizi

Zemāk (sk. 7.2.tabulu) tiek pārbaudīta mājsaimniecības dalībnieku uzaicināšanas funkcionalitāte, novērtējot piekļuves tiesību kontroli un uzaicinājumu nosūtīšanas procesa korektumu.

7.2.tabula

Mājsaimniecības lietotāja piesaistes testēšanas piemērs

Lietotāju loma:	reģistrēts lietotājs		
Prasību identifikators:	PS 2.2.2.		
Prasības apraksts			
Jānodrošina citu lietotāju pievienošanu mājsaimniecībai, izmantojot e-pasta uzaicinājumu.			
<i>Ievaddati/situācijas apraksts</i>		<i>Sagaidāmais rezultāts</i>	<i>Statuss</i>
1. Lietotājs, kas ievada datus, nav mājsaimniecības saimnieks.		nav piekļuve lietotāju pievienošanai mājsaimniecībā	pareizi
2. Lietotājs ir mājsaimniecības saimnieks, bet neievada atbilstošos ievaddatus.		netiek nosūtīts e-pasta uzaicinājums un tiek izvadīts kļūdas paziņojums	pareizi

7.2.tabulas turpinājums

<i>Ievaddati/situācijas apraksts</i>	<i>Sagaidāmais rezultāts</i>	<i>Statuss</i>
3. Lietotājs ir mājsaimniecības saimnieks un ievada atbilstošos ievaddatus, bet uzaicinātais lietotājs jau ir mājsaimniecībā.	netiek nosūtīts e-pasta uzaicinājums un tiek izvadīts kļūdas paziņojums	pareizi
4. Lietotājs ir mājsaimniecības saimnieks, ievada atbilstošos ievaddatus un uzaicinātais lietotājs nav mājsaimniecībā.	tiek nosūtīts e-pasta uzaicinājums ar atpazīšanas zīmi	pareizi

Zemāk (sk. 7.3.tabulu) tiek izvērtēta pārtikas produktu uzskaites funkcionalitāte, pārbaudot produktu pievienošanas un daudzuma atjaunošanas darbības dažādās situācijās.

7.3.tabula

Mājsaimniecības pārtikas produktu uzskaitē

Lietotāju loma:	reģistrēts lietotājs	
Prasību identifikators:	PS 2.2.3.	
Prasības apraksts		
Jānodrošina pārtikas produktu uzskaiti mājsaimniecības dalībniekiem.		
<i>Ievaddati/situācijas apraksts</i>	<i>Sagaidāmais rezultāts</i>	<i>Statuss</i>
1. Lietotājs nav mājsaimniecības dalībnieks	nav piekļuve produkta pievienošanai attiecīgajai mājsaimniecībai	pareizi
2. Lietotājs ir mājsaimniecības dalībnieks un pievieno neeksistējošu pārtikas produktu mājsaimniecībā	tiek pievienots jauns ieraksts mājsaimniecības produktu sarakstā	pareizi
3. Lietotājs ir mājsaimniecības dalībnieks un pievieno eksistējošu pārtikas produktu, bet atšķirīgu derīguma termiņu	tiek pievienots jauns ieraksts mājsaimniecības produktu sarakstā	pareizi
4. Lietotājs ir mājsaimniecības dalībnieks un pievieno eksistējošu pārtikas produktu ar vienādu derīguma termiņu	eksistējošā ieraksta produkta daudzums tiek saskaitīts ar ievadīto mājsaimniecības produktu sarakstā	pareizi

Zemāk (sk. 7.4.tabulu) tiek pārbaudīta recepšu redzamības un piekļuves kontroles funkcionalitāte, pārliecinoties, ka receptes ir pieejamas tikai tiem lietotājiem, kuriem atbilstoši noteiktajiem redzamības iestatījumiem ir piešķirtas tiesības tās apskatīt.

7.4.tabula

Receptes redzamības testēšanas piemērs

Lietotāju loma:	reģistrēts lietotājs
Prasību identifikators:	PS 2.2.5.

Prasības apraksts		
Jānodrošina receptes redzamība atbilstoši lietotāja norādījumam (privāta, mājsaimniecības ietvaros, publiska).		
<i>Ievaddati/situācijas apraksts</i>	<i>Sagaidāmais rezultāts</i>	<i>Statuss</i>
1. Receptes redzamība ir privāta, lietotājs nav receptes autors	Recepte nav pieejama lietotājam	pareizi
2. Receptes redzamība ir privāta, lietotājs izveidoja recepti	Recepte ir pieejama lietotājam	pareizi
3. Receptes redzamība ir mājsaimniecības ietvaros, lietotājs neizveidoja recepti un nav attiecīgajā mājsaimniecībā	Recepte nav pieejama lietotājam	pareizi
4. Receptes redzamība ir mājsaimniecības ietvaros, lietotājs neizveidoja recepti un ir attiecīgajā mājsaimniecībā	Recepte ir pieejama lietotājam	pareizi
5. Receptes redzamība ir mājsaimniecības ietvaros, lietotājs neizveidoja recepti un nav attiecīgajā mājsaimniecībā	Recepte ir pieejama lietotājam	pareizi

NOBEIGUMS

Kvalifikācijas darba izstrādes laikā tika izveidota mājsaimniecības pārvaldības un ēdiena recepšu meklēšanas sistēma, kas nodrošina lietotāju mājsaimniecību, pārtikas produktu un to derīguma termiņu uzskaiti, kā arī recepšu un iepirkšanās saraksta pārvaldību. Sistēmā realizēta lietotāju autentifikācija un autorizācija, iespēja veidot un uzturēt iepirkumu sarakstus, kā arī izmantot meklēšanas, filtrēšanas un kārtošanas funkcionalitāti, lai atlasītu receptes atbilstoši pieejamajiem produktiem. Tika izstrādāta arī administratora daļa, kas nodrošina sistēmas datu pārvaldību. Sistēma ir izstrādāta latviešu valodā, bet tā struktūra dot iespēju nākotnē paplašināt uz vēl citu valodu iespējām. Kā arī risinājums izstrādāts, izmantojot mūsdienīgas programmēšanas tehnoloģijas un rīkus, nodrošinot sistēmas stabilu darbību, lietojamību un paplašināmību.

Izveidotā sistēma pašlaik var tikt izmantota kā praktisks rīks mājsaimniecību ikdienas pārtikas uzskaites un recepšu meklēšanas vajadzībām. Nākotnē to iespējams attīstīt, integrējot papildu funkcionalitāti, piemēram, personalizētus recepšu ieteikumus, balstoties uz lietotāju paradumiem, produktu pievienošana izmantojot čeka pirkumus, kā arī savienojumu ar tiešsaistes veikalu platformām. Tāpat sistēmu var paplašināt līdz plašākai mājsaimniecību pārvaldības platformai vai komerciālam risinājumam, kas būtu noderīgs plašākam lietotāju lokam.

INFORMĀCIJAS AVOTI

- CubeAgency veidnis CMS videi - <https://github.com/cube-agency/filament-template> - (Resurss apskatīts 30.04.2026.)
- Filament dokumentācija - <https://filamentphp.com/docs/3.x/panels/installation> - (Resurss apskatīts 30.04.2026.)
- Inertia.js dokumentācija - <https://inertiajs.com/docs/v3/getting-started/index> - (Resurss apskatīts 01.01.2026.)
- Laravel v.12. dokumentācija - <https://laravel.com/docs/12.x> - (Resurss apskatīts 01.01.2026.)
- Mailtrap e-pasta sūtīšanas lapa - <https://mailtrap.io> - (Resurss apskatīts 03.03.2026.)
- mPDF Laravel bibliotēka PDF izdrukām - <https://mpdf.github.io/> - (Resurss apskatīts 02.06.2026.)
- OWASP organizācijas top 10 ievainojamības un kā tās novērts 2021 - <https://owasp.org/Top10/2021/> - (Resurss apskatīts 16.12.2025.)
- SASS dokumentācija - <https://sass-lang.com/documentation/> - (Resurss apskatīts 01.01.2026.)
- Vue.js v3 dokumentācija - <https://vuejs.org/guide/introduction.html> - (Resurss apskatīts 01.01.2026.)

PIELIKUMI

1.pielikums

Receptes PDF izdrukas paraugs

Laimā marinēta grilēta vista

★★★☆☆ 3.0 (2) Atsauksmes •  SiteAdmin

SAGATAVOŠANAS
LAIKS

5 min

GATAVOŠANAS
LAIKS

10 min

KOPĒJAIS LAIKS

15 min

ĒDIENREIZE Vakariņas



Sastāvdaļas

Porcijas: 4

- ☐ 750g Vistas fileja
- ☐ 2gab Laims
- ☐ 2gab Ķiploks
- ☐ 38g Brūnais cukurs
- ☐ 5g Melnie pipari (malts)
- ☐ 30ml Sojas mērce

Pagatavošanas soļi

- 1** Ar dūri (vai mīklas rulli!) izdīdiniet vistas filejas trekno galu apmēram 1,7 cm / 2/3 collas biežumā (nav nepieciešams stilbam).
- 2** evietojiet vistu un marinādi rāvējslēdzēja maisiņā, iemasējiet, lai marināde vienmērīgi sadalītos. Novietojiet uz šķīvja vai bļodas un atdzesējiet ledusskapī 24 stundas (vismaz 12 stundas, ne vairāk kā 48 stundas).
- 3** Izņemiet vistu, izmetiet marinādi. Gatavojiet, izmantojot vienu no tālāk norādītajām metodēm.
- 4** Gatavošanai (gatavošanas iekšējā temperatūra 75 °C)
- 5** PLĪTS: Uzkarsējiet eļļu lielā pannā uz vidēji lielas uguns. Cepiet vistu 3 minūtes no katras puses vai līdz tā ir karamelizēta un zeltaini brūna (skatīt video!).
- 6** GRILĒŠANA: Apsmērējiet grilu ar eļļu un uzkarsējiet līdz vidēji lielai temperatūrai (vai vidējai, ja jūsu grilēšanas zona ir spēcīga). Cepiet vistu 3 minūtes no katras puses, līdz tā ir karamelizēta (skatīt video!).
- 7** CEPEŠKRĀSNĪ: Uzkarsējiet cepeškrāsni līdz 220 °C. Cep 15 minūtes, tad uzlieciet grilu uz lielas jaudas un grilējiet 3 minūtes, lai karamelizētos virsma un pabeigtu cepšanu.
- 8** Pārlieciet vistu uz servēšanas šķīvja, brīvi pārklājiet ar foliju un atpūtiniet 3 minūtes.
- 9** Ja vēlaties, dekorējiet ar papildu koriandru/cilantro, laima šķēlītēm un čili, pēc tam pasniedziet!

Pievienošanās mājsaimniecībai uzaicinājuma e-pasta paziņojums

Temats:

Uzaicinājums pievienoties “Ģimenes” mājsaimniecībai MājasReceptes

Saturs:

Labdien,

Jūs esat uzaicināt pievienoties “Ģimenes” mājsaimniecībā! Klikšķiniet uz sekojošās saites, lai apstiprinātu piedāvājumu.

<https://majas-receptes.lv/household-invite?token=ABC123>

Paroles maiņas e-pasta paziņojums

Temats:

Paziņojums par paroles maiņu Jūsu kontam

Saturs:

Labdien,

Saņemts pieprasījums Jūsu lietotāja konta paroles maiņai.

Lai iestatītu jaunu paroli, lūdzu, izmantojiet zemāk esošo saiti:

<https://example.lv/password-reset?token=XYZ789>

Ja Jūs neesat veicis šo pieprasījumu, lūdzam nekavējoties sazināties ar sistēmas administratoru vai ignorēt šo e-pastu.

Receptes



Kārtošanas piemērs pēc atsauksmju skaita administrācijas panelā skatā

Receptes > Saraksts

Jauns Recepte

Meklēt

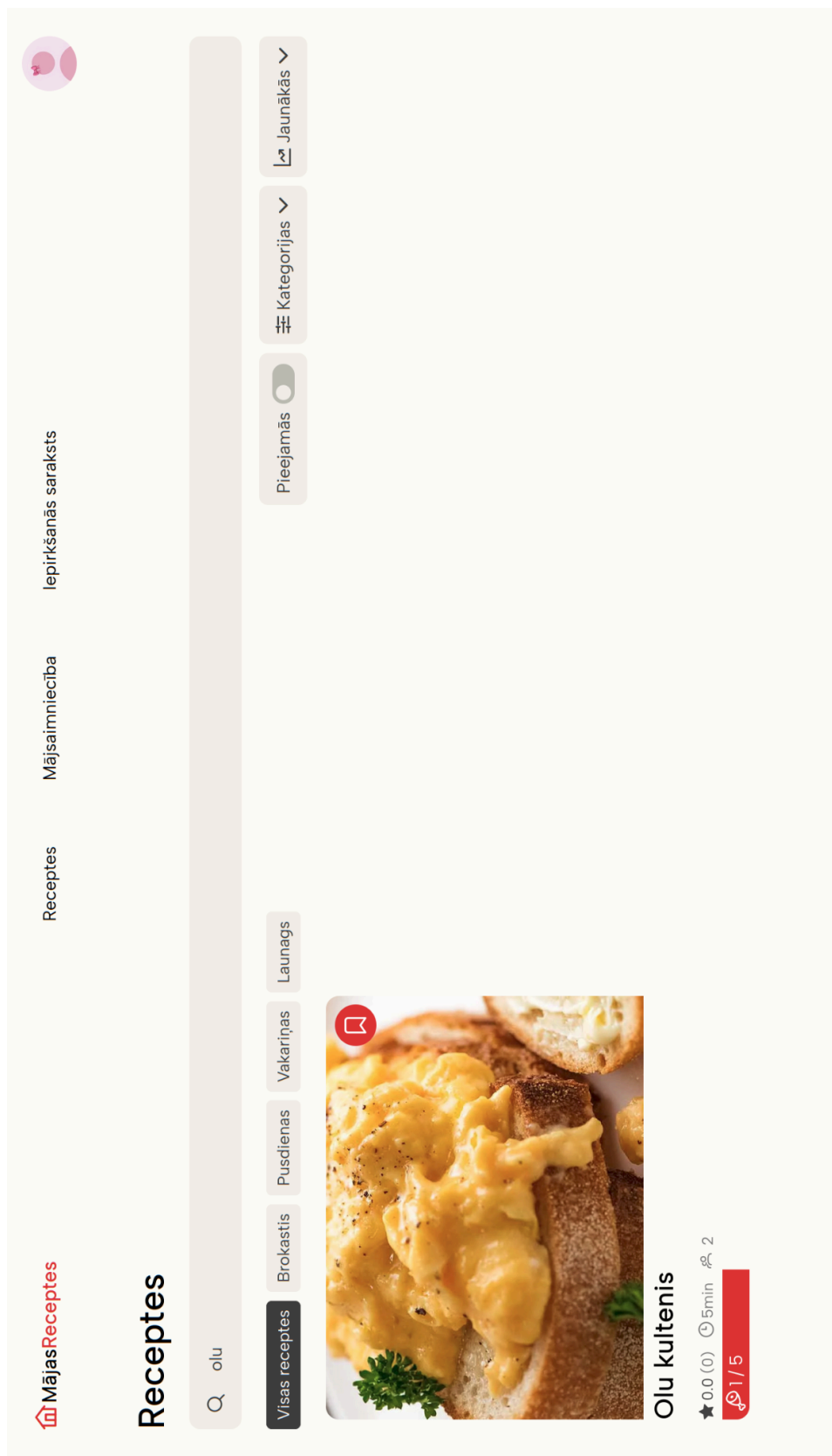
0

☐	Nosaukums	Autors	Ēdienreize	Redzamība	Porcijas	Atsauksmes	Novērtējums	Izveidots
☐	5 sastāvdaļu laša sojas bļoda	SiteAdmin	Pusdienas	private	4	0		pirms 2 nedēļām Rediģēt
☐	Olu kultenis	SiteAdmin	Brokastis	household	2	0		pirms 2 nedēļām Rediģēt
☐	Āzijas čili ķiploku garneles	SiteAdmin	Vakariņas	public	3	1	2.0	pirms 2 nedēļām Rediģēt
☐	Carbonara	SiteAdmin	Pusdienas	household	4	1	5.0	pirms 2 nedēļām Rediģēt
☐	Laimā marinēta griļēta vista	SiteAdmin	Vakariņas	public	4	2	3.0	pirms 2 nedēļām Rediģēt

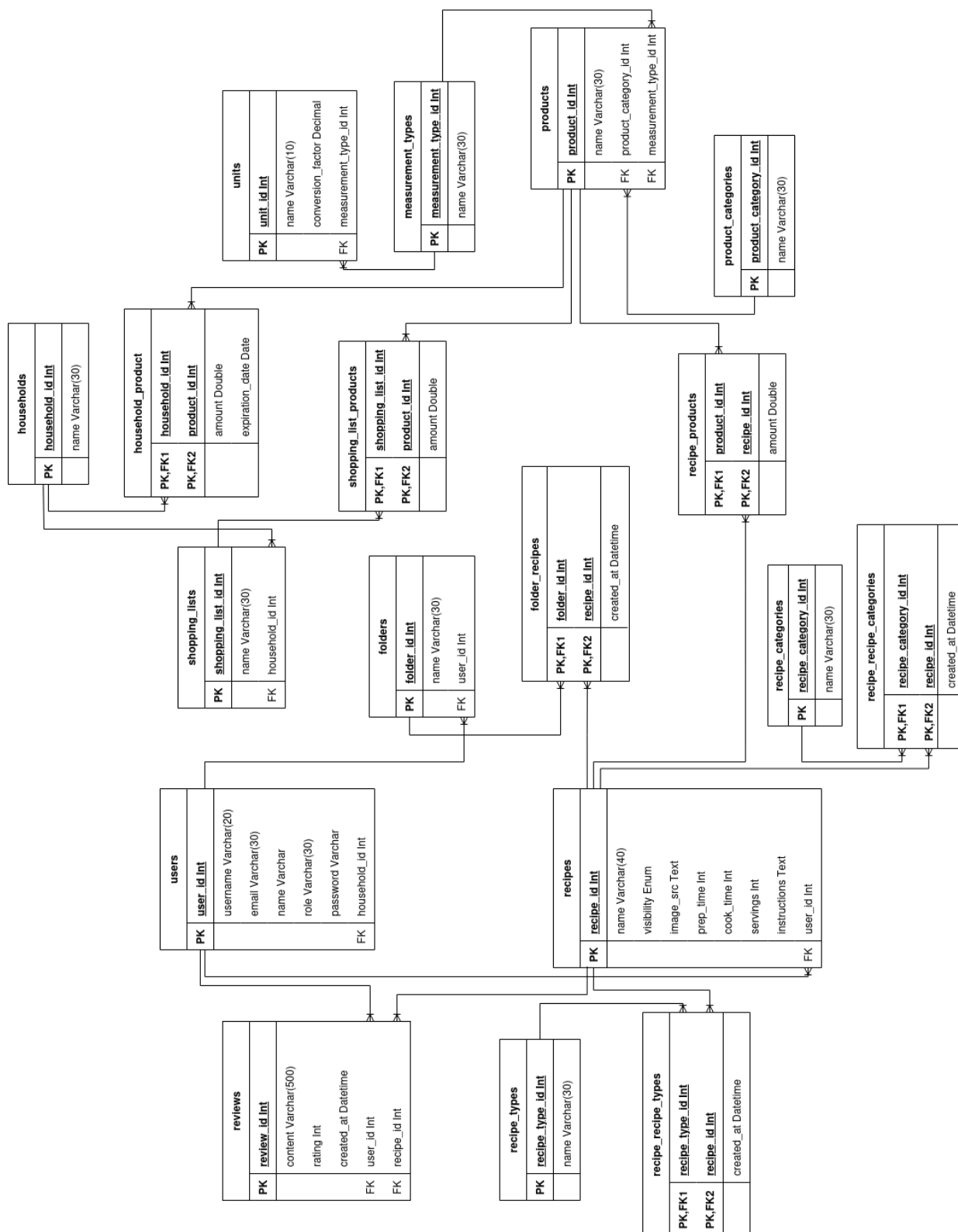
Rāda 1 līdz 5 no 5 rezultātiem

vienā lappusē10

Meklēšanas piemērs recepšu skatā



Datubāzes tabulu saišu shēma



Sistēmas pirmkods

RecipeController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\Product;
use App\Models\RecipeCategory;
use App\Models\RecipeType;
use App\Models\Unit;
use App\Services\MeasurementConversionService;
use App\Services\RecipeAvailabilityService;
use Illuminate\Http\Request;
use Illuminate\Pagination\LengthAwarePaginator;
use Illuminate\Support\Str;
use Inertia\Response;
use Inertia\Inertia;
use App\Services\PagesService;
use App\Models\Recipe;
use App\Services\BreadcrumbService;
use Illuminate\Support\Facades\Storage;
use Illuminate\Foundation\Auth\Access\AuthorizesRequests;

class RecipeController extends Controller
{
    use AuthorizesRequests;

    public function __construct(
        protected PagesService $pagesService,
        protected BreadcrumbService $breadcrumbService,
    ){}

    // -----
    // Private helpers
    // -----

    /**
     * Builds the public-facing URL for a given recipe's show page.
     */
    protected function recipeShowUrl(Recipe $recipe): string
    {
        $page = $this->pagesService->getRecipeIndexPage();

        return $page->getUrl('show', [
            'recipe' => $recipe->slug,
        ]);
    }

    /**
     * Maps a Recipe model to a flat array suitable for the frontend,
     * including availability stats relative to the authenticated user.
     */
    private function mapRecipe(Recipe $recipe, $user): array
```

```

{
    $availability = RecipeAvailabilityService::calculate($recipe, $user);

    return [
        'id'                => $recipe->id,
        'slug'              => $recipe->slug,
        'name'              => $recipe->name,
        'image_src'         => $recipe->image_src,
        'prep_time'         => $recipe->prep_time,
        'cook_time'         => $recipe->cook_time,
        'total_time'        => $recipe->total_time,
        'servings'          => $recipe->servings,
        'average_rating'    => $recipe->average_rating,
        'reviews_count'     => $recipe->reviewsCount,
        'missing_products_count' => $availability['missing_products_count'],
        'available_products_count' => $availability['available_products_count'],
        'total_products_count' => $availability['total_products_count'],
        'compatibility'     => $availability['compatibility'],
        'url'               => $this->recipeShowUrl($recipe),
    ];
}

/**
 * Loads and maps the authenticated user's folders with a thumbnail
 * (first recipe image) and a recipe count.
 *
 * Returns an empty array when no user is authenticated, since guests have no
 * folders.
 */
private function mapFolders($user): array
{
    if (!$user) {
        return [];
    }

    return $user->folders()
        ->with(['recipes:id,image_src'])
        ->get()
        ->map(fn($folder) => [
            'id'                => $folder->id,
            'name'              => $folder->name,
            'thumbnail'         => $folder->recipes->first()?->image_src,
            'recipe_count'      => $folder->recipes->count(),
        ])
        ->toArray();
}

/**
 * Applies a named sort order to the query and paginates the result (12 per page).
 *
 * Supported sort values: 'highest_rated', 'most_reviewed', 'quickest', default →
 * newest.
 */
private function applySorting($query, string $sort, Request $request)
{
    $this->applySortOrder($query, $sort);

    return $query->paginate(12)->withQueryString();
}

```

```

/**
 * Applies a named sort order to the query without paginating.
 * Used when in-memory filtering (e.g. availability) must happen before slicing.
 */
private function applySortingWithoutPaginate($query, string $sort)
{
    return $this->applySortOrder($query, $sort);
}

/**
 * Mutates the query in-place with the requested sort clause and returns it.
 * Extracted to avoid duplicating the match expression across the two sorting
helpers.
 */
private function applySortOrder($query, string $sort)
{
    match ($sort) {
        'highest_rated' => $query->withAvg('reviews',
'rating')->orderByDesc('reviews_avg_rating'),
        'most_reviewed' =>
$query->withCount('reviews')->orderByDesc('reviews_count'),
        'quickest' => $query->orderByRaw('(prep_time + cook_time) ASC'),
        default => $query->latest(),
    };

    return $query;
}

/**
 * Transforms a recipe_products collection in-place, converting each
 * product's stored base amount back to a human-readable unit.
 *
 * Shared between show() and edit() to avoid duplication.
 */
private function transformRecipeProducts($recipeProducts): void
{
    $recipeProducts->transform(function ($recipeProduct) {
        $converted = MeasurmentConversionService::fromBaseAmount(
            $recipeProduct->amount,
            $recipeProduct->product
        );

        return [
            'id' => $recipeProduct->id,
            'amount' => $converted['amount'],
            'product' => [
                'id' => $recipeProduct->product_id,
                'name' => $recipeProduct->product->name,
            ],
            'unit' => [
                'id' => $converted['unit_id'],
                'name' => $converted['unit'],
            ],
        ];
    });
}

/**

```



```

    * Stores an uploaded recipe image and removes the old file when replacing.
    * Returns the new storage path, or $currentPath when no new file was uploaded.
    */
    private function handleImageUpload(Request $request, ?string $currentPath):
?string
    {
        if (!$request->hasFile('image_src')) {
            return $currentPath;
        }

        // Remove the previous image to avoid orphaned files on disk.
        if ($currentPath) {
            Storage::disk('public')->delete($currentPath);
        }

        return $request->file('image_src')->store('recipes', 'public');
    }

    /**
     * Replaces all recipe_products for a recipe using the validated input.
     * Converts each product amount from the user-selected unit to the base unit.
     */
    private function syncRecipeProducts(Recipe $recipe, array $recipeProductsData):
void
    {
        // Delete existing products before re-inserting to keep the relation clean.
        $recipe->recipeProducts()->delete();

        foreach ($recipeProductsData as $item) {
            $unit = Unit::findOrFail($item['unit_id']);
            $product = Product::findOrFail($item['product_id']);

            $amountInBase = MeasurmentConversionService::toBaseAmount($item['amount'],
$unit, $product);

            $recipe->recipeProducts()->create([
                'product_id' => $item['product_id'],
                'amount'      => $amountInBase,
            ]);
        }
    }

    /**
     * Validation rules shared between store() and update().
     * The image rule is stricter when a new file is being uploaded.
     */
    private function recipeValidationRules(Request $request): array
    {
        return [
            'name' => 'required|string',
            'image_src' => $request->hasFile('image_src')
                ? 'nullable|image|max:2048'
                : 'nullable',
            'visibility' => 'required|string',
            'prep_time' => 'required|numeric',
            'cook_time' => 'required|numeric',
            'servings' => 'required|numeric',
            'instructions' => 'required|array',
            'instructions.*' => 'required|string',
        ];
    }

```

```

        'recipe_products'                => 'required|array',
        'recipe_products.*.product_id'   => 'required|numeric',
        'recipe_products.*.amount'       => 'required|numeric|min:0',
        'recipe_products.*.unit_id'      => 'required|numeric',
        'recipe_type_id'                 => 'nullable|exists:recipe_types,id',
        'recipe_category_ids'            => 'nullable|array',
        'recipe_category_ids.*'          => 'exists:recipe_categories,id',
    ];
}

/**
 * Generates a unique slug for a recipe by appending an incrementing suffix
 * if the base slug (username + recipe name) is already taken.
 *
 * Example: "john-pasta" → "john-pasta-2" → "john-pasta-3"
 */
private function generateUniqueSlug(string $username, string $recipeName): string
{
    $baseSlug = $username . '-' . Str::slug($recipeName);
    $slug      = $baseSlug;
    $counter   = 2;

    while (Recipe::where('slug', $slug)->exists()) {
        $slug = $baseSlug . '-' . $counter++;
    }

    return $slug;
}

// -----
// Public actions (CRUD + extras)
// -----

/**
 * Displays the public recipe listing with filtering, sorting, and optional
 * "available ingredients only" mode that requires in-memory post-filtering.
 */
public function index(Request $request): Response
{
    $page = $this->pagesService->getRecipeIndexPage();
    $user = auth()->user();

    $sort          = $request->get('sort', 'newest');
    $availableOnly = $request->boolean('available') && $user;
    $categoryIds   = array_filter(explode(',', $request->get('categories', '')));

    // Base query: only columns needed by mapRecipe() to keep the payload lean.
    $query = Recipe::select('id', 'name', 'image_src', 'slug', 'prep_time',
'cook_time', 'servings')
        ->visibleTo($user)
        ->when($request->search, fn($q, $s) => $q->where('name', 'like',
"%{$s}%"));
    ->when($request->type, fn($q, $id) => $q->where('recipe_type_id', $id))
    ->when($categoryIds, function ($q) use ($categoryIds) {
        // Each category must match (AND logic), so we add a whereHas per ID.
        foreach ($categoryIds as $categoryId) {
            $q->whereHas('recipeCategories', fn($q) =>
$q->where('recipe_categories.id', $categoryId));
        }
    });
}

```

```

    });

    if ($availableOnly) {
        // Availability is computed in PHP, so we must fetch all matching records
        // first, filter them in memory, then manually paginate the result.
        // Note: this is a performance concern on large datasets; consider moving
        // availability filtering to the DB layer if recipe count grows
        significantly.
        $perPage = 12;
        $currentPage = $request->get('page', 1);

        $allRecipes = $this->applySortingWithoutPaginate($query, $sort)
            ->get()
            ->map(fn($recipe) => $this->mapRecipe($recipe, $user))
            ->filter(fn($recipe) => $recipe['compatibility'] == 100)
            ->values();

        $recipes = new LengthAwarePaginator(
            $allRecipes->forPage($currentPage, $perPage),
            $allRecipes->count(),
            $perPage,
            $currentPage,
            ['path' => $request->url(), 'query' => $request->query()]
        );
    } else {
        // Standard DB-level sort + paginate path.
        $recipes = $this->applySorting($query, $sort, $request)
            ->through(fn($recipe) => $this->mapRecipe($recipe, $user));
    }

    return Inertia::render('Recipe/Index', [
        'page_name' => $page->name,
        'blocks' => json_decode($page->blocks) ?? [],
        'recipes' => $recipes,
        'folders' => $this->mapFolders($user),
        'categories' => RecipeCategory::all(),
        'types' => RecipeType::all(),
        'filters' => [
            'search' => $request->search,
            'sort' => $sort,
            'categories' => $request->get('categories', ''),
            'available' => $availableOnly,
        ],
    ]);
}

/**
 * Displays only the recipes created by the authenticated user.
 */
public function myRecipes(Request $request): Response
{
    $page = $this->pagesService->getRecipeIndexPage();
    $user = auth()->user();

    // Count is fetched from a fresh query to avoid any state left by paginate().
    $totalCount = Recipe::where('user_id', $user->id)->count();

    $recipes = Recipe::where('user_id', $user->id)

```

```

        ->select('id', 'name', 'image_src', 'slug', 'prep_time', 'cook_time',
'servings')
        ->when($request->search, fn($q, $s) => $q->where('name', 'like',
"%{$s}%"))
        ->paginate(12)
        ->withQueryString()
        ->through(fn($recipe) => $this->mapRecipe($recipe, $user));

    return Inertia::render('Recipe/MyRecipes', [
        'page_name'    => $page->name,
        'blocks'       => json_decode($page->blocks) ?? [],
        'recipes'      => $recipes,
        'recipe_count' => $totalCount,
        'filters'      => ['search' => $request->search],
        'folders'      => $this->mapFolders($user),
    ]);
}

/**
 * Renders the recipe creation form with all supporting lookups.
 */
public function create(): Response
{
    return Inertia::render('Recipe/Create', [
        'categories' => RecipeCategory::all(),
        'types'      => RecipeType::all(),
        'products'   => Product::all(),
        'units'      => Unit::all(),
        'breadcrumbs' => $this->breadcrumbService->forRecipeCreate(),
    ]);
}

/**
 * Displays a single recipe.
 */
public function show(Request $request, Recipe $recipe): Response
{
    $recipe->load([
        'user:id,username,avatar_src',
        'recipeProducts.product.measurementType.units',
        'recipeType',
        'recipeCategories',
    ]);

    // Convert stored base amounts to display units for each ingredient.
    $this->transformRecipeProducts($recipe->recipeProducts);

    $reviews = $recipe->reviews()
        ->with('user:id,username,avatar_src')
        ->latest()
        ->paginate(5);

    $user      = auth()->user();
    $fromFolder = $request->query('from_folder')
        ? $user->folders()->find($request->query('from_folder'))
        : null;

    return Inertia::render('Recipe/Show', [
        'recipe' => $recipe,

```

```

        'reviews'      => $reviews,
        'url'          => $this->recipeShowUrl($recipe),
        'breadcrumbs' => $fromFolder
            ? $this->breadcrumbService->forRecipeFromFolder($recipe, $fromFolder)
            : $this->breadcrumbService->forRecipe($recipe),
        'folders'      => $this->mapFolders($user),
    ]]);
}

/**
 * Renders the recipe edit form pre-populated with existing data.
 */
public function edit(Recipe $recipe): Response
{
    $recipe->load([
        'recipeProducts.product.measurementType.units',
        'recipeType',
        'recipeCategories',
    ]);

    // Convert stored base amounts back to human-readable units for the form.
    $this->transformRecipeProducts($recipe->recipeProducts);

    return Inertia::render('Recipe/Edit', [
        'recipe'      => $recipe,
        'products'    => Product::all(),
        'units'       => Unit::all(),
        'types'       => RecipeType::all(),
        'categories'  => RecipeCategory::all(),
        'breadcrumbs' => $this->breadcrumbService->forRecipeEdit($recipe),
    ]);
}

/**
 * Persists updates to an existing recipe.
 */
public function update(Request $request, Recipe $recipe)
{
    $data = $request->validate($this->recipeValidationRules($request));

    $imagePath = $this->handleImageUpload($request, $recipe->image_src);

    $recipe->update([
        'name'          => $data['name'],
        'image_src'     => $imagePath,
        'visibility'    => $data['visibility'],
        'prep_time'     => $data['prep_time'],
        'cook_time'     => $data['cook_time'],
        'servings'      => $data['servings'],
        'instructions'  => $data['instructions'],
        'recipe_type_id' => $data['recipe_type_id'] ?? null,
    ]);

    $recipe->recipeCategories()->sync($data['recipe_category_ids'] ?? []);

    $this->syncRecipeProducts($recipe, $data['recipe_products']);

    return redirect()
        ->to($this->recipeShowUrl($recipe))

```

```

        ->with('success', 'Recepte informācija veiksmīgi atjaunināta');
    }

/**
 * Creates and persists a new recipe owned by the authenticated user.
 */
public function store(Request $request)
{
    $data = $request->validate($this->recipeValidationRules($request));

    $imagePath = $this->handleImageUpload($request, null);

    $user = auth()->user();

    // Slug is username + URL-safe recipe name, with a numeric suffix added
    // if a recipe with the same slug already exists.
    $slug = $this->generateUniqueSlug($user->username, $data['name']);

    $recipe = Recipe::create([
        'name'          => $data['name'],
        'image_src'     => $imagePath,
        'slug'          => $slug,
        'visibility'    => $data['visibility'],
        'prep_time'     => $data['prep_time'],
        'cook_time'     => $data['cook_time'],
        'servings'      => $data['servings'],
        'instructions'  => $data['instructions'],
        'user_id'       => $user->id,
        'recipe_type_id' => $data['recipe_type_id'] ?? null,
    ]);

    $recipe->recipeCategories()->sync($data['recipe_category_ids'] ?? []);

    $this->syncRecipeProducts($recipe, $data['recipe_products']);

    return redirect()
        ->to($this->recipeShowUrl($recipe))
        ->with('success', 'Recepte ir veiksmīgi izveidota');
}

/**
 * Deletes a recipe along with its image and related products.
 */
public function destroy(Recipe $recipe)
{
    // Remove the stored image file to avoid orphaned files on disk.
    if ($recipe->image_src) {
        Storage::disk('public')->delete($recipe->image_src);
    }

    $recipe->recipeProducts()->delete();
    $recipe->delete();

    return redirect()
        ->route('recipe.my')
        ->with('success', 'Recepte ir veiksmīgi dzēsta');
}
}

```

RecipeAvailabilityService.php

```
<?php

namespace App\Services;

use App\Models\Recipe;
use App\Models\User;

class RecipeAvailabilityService
{
    /**
     * Calculates how many of a recipe's required products are available
     * in the user's active household, and returns a compatibility percentage.
     *
     * Returns all-zero / 0% compatibility when the user has no active household.
     */
    public static function calculate(Recipe $recipe, User $user): array
    {
        $recipe->load('recipeProducts.product');

        $total = $recipe->recipeProducts->count();

        $household = $user->activeHousehold();

        if (!$household) {
            return self::unavailableResult($total);
        }

        $household->load('householdProducts.product');

        // Key by product_id for O(1) lookups inside the loop.
        $householdProducts = $household->householdProducts->keyBy('product_id');

        $available = 0;

        foreach ($recipe->recipeProducts as $recipeProduct) {
            $householdProduct = $householdProducts->get($recipeProduct->product_id);

            // A product counts as available only when the household holds
            // at least the required amount (exact match is acceptable).
            $isAvailable = $householdProduct
                && $householdProduct->amount >= $recipeProduct->amount;

            if ($isAvailable) {
                $available++;
            }
        }

        $missing = $total - $available;

        return [
            'available_products_count' => $available,
            'missing_products_count'   => $missing,
            'total_products_count'     => $total,
            'compatibility'             => $total > 0 ? ($available / $total) * 100 :
0,
        ];
    }
}
```

```

}

/**
 * Returns a zeroed-out result for when availability cannot be determined
 * (e.g. the user has no active household).
 */
private static function unavailableResult(int $total): array
{
    return [
        'available_products_count' => 0,
        'missing_products_count'   => $total,
        'total_products_count'     => $total,
        'compatibility'            => 0,
    ];
}
}

```


MeasurementConversionService.php

```
<?php

namespace App\Services;

use App\Models\Product;
use App\Models\Unit;

class MeasurementConversionService
{
    /**
     * Spoon-like units (tablespoon / teaspoon) that are excluded when selecting
     * a display unit, because they are ambiguous for non-volume measurement types.
     */
    private const VOLUME_EXCLUDED_UNITS = ['ēdmk.', 'tējk.'];

    /**
     * Converts a human-entered amount (in the given unit) to the product's
     * canonical base amount for storage.
     *
     * Throws if the unit belongs to a different measurement type than the product,
     * e.g. trying to store a weight product using a volume unit.
     */
    public static function toBaseAmount(float $amount, Unit $unit, Product $product):
float
    {
        if ($unit->measurement_type_id !== $product->measurement_type_id) {
            throw new \InvalidArgumentException(
                'Unit is not compatible with this product.'
            );
        }

        return $amount * $unit->conversion_factor;
    }

    /**
     * Converts a stored base amount back to the most human-readable unit
     * available for the given product.
     *
     * The selected unit is the largest one whose conversion factor still fits
     * into the base amount (e.g. 1500 g → 1.5 kg). Spoon units are excluded
     * from consideration to prevent nonsensical displays for non-liquid products.
     *
     * Returns an array with keys: 'amount', 'unit' (name), 'unit_id'.
     */
    public static function fromBaseAmount(float $baseAmount, Product $product): array
    {
        // Fetch all units for this product's measurement type, largest factor first.
        $units = $product->measurementType
            ->units
            ->sortByDesc('conversion_factor');

        $units = self::excludeSpoonUnits($units);

        // Pick the largest unit that still produces a whole-or-decimal value ≥ 1.
        // Falls back to the smallest available unit if nothing fits (e.g. tiny
        amounts).
    }
}
```

```

$unit = $units->first(fn($unit) => $baseAmount >= $unit->conversion_factor)
?: $units->last();

$amount = round($baseAmount / $unit->conversion_factor, 2);

return [
    'amount' => $amount,
    'unit'    => $unit->name,
    'unit_id' => $unit->id,
];
}

/**
 * Removes spoon units (and any unit sharing their conversion factor) from
 * the candidate collection so they are never chosen as a display unit.
 *
 * Filtering by conversion_factor rather than name guards against units that
 * are stored under a slightly different label but represent the same ratio.
 */
private static function excludeSpoonUnits($units)
{
    $excludedFactors = $units
        ->whereIn('name', self::VOLUME_EXCLUDED_UNITS)
        ->pluck('conversion_factor');

    if ($excludedFactors->isEmpty()) {
        return $units;
    }

    return $units->filter(
        fn($unit) => !$excludedFactors->contains($unit->conversion_factor)
    );
}
}

```

Recipe.php

```
<?php

namespace App\Models;

use App\Enums\Recipe\Visibility;
use App\Models\RecipeProduct;
use App\Services\PagesService;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Recipe extends Model
{
    protected $fillable = [
        'slug',
        'name',
        'visibility',
        'image_src',
        'content',
        'prep_time',
        'cook_time',
        'servings',
        'instructions',
        'product_id',
        'user_id',
        'recipe_type_id',
    ];

    protected $casts = [
        'image_src' => 'string',
        'ingredients' => 'array',
        'instructions' => 'array',
        'visibility' => Visibility::class,
    ];

    /**
     * Appended virtual attributes computed on every serialisation.
     * Keep this list short – each entry triggers an extra query if not eager-loaded.
     */
    protected $appends = [
        'total_time',
        'average_rating',
        'reviews_count',
    ];

    // -----
    // Relationships
    // -----

    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }

    public function reviews(): HasMany
```

```

{
    return $this->hasMany(Review::class);
}

public function folders(): BelongsToMany
{
    return $this->belongsToMany(Folder::class);
}

public function recipeProducts(): HasMany
{
    return $this->hasMany(RecipeProduct::class);
}

public function recipeCategories(): BelongsToMany
{
    return $this->belongsToMany(RecipeCategory::class);
}

public function recipeType(): BelongsTo
{
    return $this->belongsTo(RecipeType::class);
}

// -----
// Accessors
// -----

/**
 * Sum of prep and cook time in minutes.
 */
public function getTotalTimeAttribute(): int
{
    return ($this->prep_time ?? 0) + ($this->cook_time ?? 0);
}

/**
 * Average review rating rounded to one decimal, or null when no reviews exist.
 */
public function getAverageRatingAttribute(): ?float
{
    $avg = $this->reviews()->avg('rating');

    return $avg !== null ? round((float) $avg, 1) : null;
}

/**
 * Total number of reviews for this recipe.
 */
public function getReviewsCountAttribute(): int
{
    return $this->reviews()->count();
}

// -----
// Scopes
// -----

/**

```

```

* Limits the query to recipes the given user is allowed to see.
*
* Visibility rules:
* - Public, visible to everyone, including guests (null user).
* - Private, visible only to the recipe's author.
* - Household, visible to any member of a household the author belongs to.
*/
public function scopeVisibleTo(Builder $query, ?User $user): Builder
{
    // Guests may only see public recipes.
    if ($user === null) {
        return $query->where('visibility', Visibility::Public);
    }

    return $query->where(function (Builder $q) use ($user) {
        $q->where('visibility', Visibility::Public)

        ->orWhere(function (Builder $q) use ($user) {
            // Private: only the author themselves.
            $q->where('visibility', Visibility::Private)
                ->where('user_id', $user->id);
        })

        ->orWhere(function (Builder $q) use ($user) {
            // Household: the recipe's author must share at least one
            // household with the viewing user.
            $q->where('visibility', Visibility::Household)
                ->whereHas('user.households', function (Builder $q) use
($user) {
                    $q->whereIn(
                        'households.id',
                        $user->households()->pluck('households.id')
                    );
                });
        });
    });
}

```

HouseholdProductController.php

```
<?php

namespace App\Http\Controllers;

use App\Models\HouseholdProduct;
use App\Models\Product;
use App\Models\Unit;
use App\Services\MeasurmentConversionService;
use Illuminate\Http\RedirectResponse;
use Illuminate\Http\Request;

class HouseholdProductController extends Controller
{
    /**
     * Adds a product to the authenticated user's active household.
     *
     * If the same product already exists in the household with a matching
     * expiration date, the stored amount is incremented rather than creating
     * a duplicate entry. Different expiration dates are treated as separate
     * stock items (e.g. a fresh batch alongside an older one).
     *
     * All amounts are converted to the product's base unit before storage.
     */
    public function store(Request $request): RedirectResponse
    {
        $validated = $request->validate([
            'product_id'      => 'required|exists:products,id',
            'amount'          => 'required|numeric',
            'unit_id'         => 'required|exists:units,id',
            'expiration_date' => 'nullable|date',
        ]);

        $household = auth()->user()->activeHousehold();
        $unit      = Unit::findOrFail($validated['unit_id']);
        $product   = Product::findOrFail($validated['product_id']);

        $amountInBaseUnit = MeasurmentConversionService::toBaseAmount(
            $validated['amount'],
            $unit,
            $product
        );

        $existingProduct = $household->householdProducts()
            ->where('product_id', $product->id)
            ->first();

        $sameExpirationDate = $existingProduct
            && $existingProduct->expiration_date == $validated['expiration_date'];

        if ($sameExpirationDate) {
            // Merge into the existing stock entry rather than creating a duplicate.
            $existingProduct->increment('amount', $amountInBaseUnit);
        } else {
            $household->householdProducts()->create([
                'product_id'      => $product->id,
                'amount'          => $amountInBaseUnit,
            ]);
        }
    }
}
```

```

        'expiration_date' => $validated['expiration_date'],
    ]]);
    }
    return back()->with('success', 'Produkts veiksmīgi pievienots');
}

/**
 * Replaces the amount and expiration date of an existing household product.
 *
 * The incoming amount is converted from the user-selected unit to the
 * product's base unit before being stored.
 */
public function update(Request $request, HouseholdProduct $householdProduct):
RedirectResponse
{
    $validated = $request->validate([
        'amount'          => 'required|numeric',
        'unit_id'          => 'required|exists:units,id',
        'expiration_date' => 'nullable|date',
    ]]);

    $unit = Unit::findOrFail($validated['unit_id']);
    $product = $householdProduct->product;

    $amountInBaseUnit = MeasurmentConversionService::toBaseAmount(
        $validated['amount'],
        $unit,
        $product
    );
    $householdProduct->update([
        'amount'          => $amountInBaseUnit,
        'expiration_date' => $validated['expiration_date'],
    ]]);
    return back()->with('success', 'Produkta informācija veiksmīgi atjaunota');
}

/**
 * Removes a product entry from the household.
 */
public function destroy(HouseholdProduct $householdProduct): RedirectResponse
{
    $householdProduct->delete();
    return back()->with('success', 'Produkts veiksmīgi dzēsts no mājsaimniecības');
}
}

```

HouseholdEmailController.php

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use App\Models\Household;
use App\Models\HouseholdInvitation;
use App\Mail\HouseholdInviteMail;
use Illuminate\Support\Facades\Mail;
use Illuminate\Support\Str;
use Inertia\Inertia;
use App\Services\HouseholdUrlService;

class HouseholdEmailInviteController extends Controller
{
    public function __construct(protected HouseholdUrlService $householdUrlService) {}

    /**
     * Send an email invitation to join a household.
     * Only the household owner can send invitations.
     */
    public function send(Request $request, Household $household)
    {
        // Verify the authenticated user is the household owner
        $isOwner = $household->users()
            ->where('user_id', auth()->id())
            ->where('household_user.role', 'owner')
            ->exists();

        abort_if(!$isOwner, 403);

        $request->validate(['email' => 'required|email']);

        $email = $request->email;

        // Prevent inviting someone who is already a member
        $alreadyMember = $household->users()
            ->where('email', $email)
            ->exists();

        if ($alreadyMember) {
            return back()->withErrors(['email' => 'Lietotājs jau ir māsaimniecībā']);
        }

        // Create a new invitation or refresh an existing one for this email,
        // resetting the token and expiry in case it was previously sent
        $invitation = HouseholdInvitation::updateOrCreate(
            ['household_id' => $household->id, 'email' => $email],
            [
                'invited_by' => auth()->id(),
                'token' => Str::random(64),
                'expires_at' => now()->addDays(7),
                'accepted_at' => null,
            ]
        );
        $invitation->load(['household', 'inviter']);
    }
}
```



```

    Mail::to($email)->send(new HouseholdInviteMail($invitation));

    return back()->with('success', "Uzaicinājums nosūtīts {$email}!");
}

/**
 * Display the invitation acceptance page for a given token.
 * Passes login/email-match state to the frontend to render the correct UI.
 */
public function show(string $token)
{
    $invitation = HouseholdInvitation::with(['household', 'inviter'])
        ->where('token', $token)
        ->firstOrFail();

    // Reject expired or already-accepted invitations
    abort_if(!$invitation->isValid(), 410, 'Šis uzaicinājums ir beidzies vai jau izmantots');

    // Store the intended URL so unauthenticated users are redirected back here after login
    if (!auth()->check()) {
        session(['url.intended' => route('households.invite.email.show', $token)]);
    }

    return Inertia::render('Household/AcceptInvite', [
        'invitation' => [
            'token' => $token,
            'household' => $invitation->household->only('name'),
            'inviter' => $invitation->inviter->only('name'),
            'email' => $invitation->email,
        ],
        'isLoggedIn' => auth()->check(),
        // True only if the logged-in user's email matches the invitation recipient
        'emailMatches' => auth()->check() && auth()->user()->email === $invitation->email,
    ]);
}

/**
 * Accept the invitation and attach the authenticated user to the household.
 */
public function accept(string $token)
{
    $invitation = HouseholdInvitation::with('household')
        ->where('token', $token)
        ->firstOrFail();

    abort_if(!$invitation->isValid(), 410, 'Šis uzaicinājums ir beidzies vai jau izmantots');

    $user = auth()->user();

    // Enforces that the logged-in user is actually the intended recipient
    abort_if($user->email !== $invitation->email, 403, 'Šis uzaicinājums nav paredzēts jums');

    $user = auth()->user();

```

```

    $household = $invitation->household;

    // Only attach the user if they aren't already a member (e.g. edge case /
double submit)
    $alreadyMember = $household->users()->where('user_id', $user->id)->exists();

    if (!$alreadyMember) {
        $household->users()->attach($user->id, ['role' => 'member']);
    }

    // Mark the invitation as accepted so it can't be reused
    $invitation->update(['accepted_at' => now()]);

    // Redirect to the household page with a success message
    return redirect(
        $this->householdUrlService->showUrl($user)
    )->with('success', "Sveicināti '{$household->name}'!");
}

/**
 * Cancel (delete) a pending invitation.
 * Only the household owner can cancel invitations.
 */
public function cancel(Household $household, HouseholdInvitation $invitation)
{
    abort_if(auth()->id() !== $household->owner_id, 403);
    $invitation->delete();
    return back()->with('success', 'Ielūgums atcelts');
}
}

```